Graph-Structured Models, Their Algorithms, and the Properties That Referee Them

M. J. Wilson

January 1, 2025

Abstract

A self-contained account of the three problem classes this project supports, the algorithms that solve them, and—the part a methods section usually omits—the mathematical properties each algorithm is validated against, and why those properties are the right referees. Written for a reader with a scientific and performance-computing background but no background in phylogenetics or statistical physics. It describes *algorithms*, not an implementation: no result here depends on how any of it is coded, and the companion paper carries the results themselves.

Contents

1 Notation **2**

2 The Shape All Four Problems Share **3**
2.1 Factor Graphs: The One Structure . . . 3
2.2 The Continuous Optimization Abstraction . . . 4
2.3 Sampling the Posterior, and Adapting the Sampler . . . 5
2.4 The Discrete Search as a Decision Process . . . 6
2.5 Actor–Critic and Proximal Policy Optimization . . . 7
2.6 Planning with a Learned Prior and Value . . . 7
2.7 Surrogates and Bounds . . . 8

3 Problem Statement: Phylogenetic Inference **9**
3.1 The model . . . 9
3.2 As a factor graph . . . 9
3.3 The algorithm: simulation . . . 9
3.4 The algorithm: pruning . . . 9
3.5 The algorithm: discrete search . . . 11

4 Problem Statement: Potts Models in an External Field **12**
4.1 The model . . . 12
4.2 As a factor graph . . . 12
4.3 Why an odd cycle is the whole story . . . 12
4.4 The algorithm: belief propagation . . . 13
4.5 The algorithm: ground states as cuts . . . 13
4.6 The algorithm: alpha expansion . . . 14
4.7 The algorithm: sampling . . . 14
4.8 Temperature, annealing and tempering . . . 15

5 Problem Statement: Hidden Markov Models **15**
5.1 The model . . . 15
5.2 As a factor graph . . . 16
5.3 The algorithm: forward–backward . . . 16
5.4 The algorithm: expectation-maximization . . . 17
5.5 Two decodings, and they are different answers . . . 17

6 Problem Statement: The Coupled Spatio-Sequential Model **17**
6.1 The model . . . 17
6.2 As a factor graph . . . 18
6.3 The estimator: block-coordinate ascent . . . 18
6.4 The initializer: data-sampled burn-in . . . 19

7 Problem Statement: LDPC Decoding **20**
7.1 The model . . . 20
7.2 As a factor graph . . . 21
7.3 The algorithm: sum-product in the log domain . . . 21
7.4 The all-zero codeword . . . 21

8 The Properties That Referee Each Method **22**
8.1 Exact answers, where an oracle exists . . . 22
8.2 Structural invariants, where no oracle reaches . . . 22
8.3 Recovery, which is the acceptance test for a fit . . . 23
8.4 Where the likelihood has no maximum, or no curvature . . . 23
8.5 Agreement across implementations is a tolerance . . . 23
8.6 Published constants, and asymptotic results . . . 23

9 Relevance to the Roadmap **23**

A Derivations Cited From the Text **29**
A.1 Pruning is the marginalization over internal states . . . 29
A.2 Forward–backward is the same marginalization on a chain . . . 30
A.3 Sum-product on a tree, and its loopy stationary point . . . 30
A.4 The Bethe free energy is stationary at a sum-product fixed point . . . 31
A.5 Detailed balance for a cluster move in a field . . . 31
A.6 The exchange ratio in parallel tempering . . . 31
A.7 Density evolution on the erasure channel . . . 31
A.8 The delta method for an interval at a fit . . . 32

B Derivations for the Actor–Critic **32**

C Derivations of the Bounds **33**

D Reference Taxonomy **34**

1 Notation

One symbol, one meaning, across this document and the companion paper.

Symbol	Meaning
$\mathcal{T}$	a tree topology
$\mathcal{G}$	a general undirected graph
$\mathcal{A}$	the state alphabet, $\lvert\mathcal{A}\rvert = k$
ρ	the root of a rooted tree
$\mathcal{N}(\cdot)$	the neighbourhood a discrete move set reaches in one step
$\mathbf{Q}$	a rate matrix; $\mathbf{P}(t) = \exp(\mathbf{Q}t)$ its transition matrix
$\mathbf{S}$	a symmetric exchangeability matrix
$\boldsymbol{\pi}$	a stationary or initial distribution
$\boldsymbol{\tau}$	branch lengths
$\boldsymbol{\theta}$	unconstrained parameters, $\boldsymbol{\theta} \in \mathbb{R}^d$
ϕ	the constraint map, $\phi(\boldsymbol{\theta})$ the feasible parameters
$\mathbf{D}$	the observed data
$\mathcal{L}$	the scalar being minimized
H	a Hamiltonian
$\mathbf{Z}$	a hidden state path
$\mathcal{E}$	an emission family
x_i	a discrete variable of a factor graph, $x_i \in \mathcal{A}$
ψ_a	a factor: a non-negative function of the variables $x_{\partial a}$ it touches
$m_{a\to i},\ m_{i\to a}$	a message from factor to variable, and from variable to factor
$b_i,\ b_a$	a belief: the marginal of a variable or a factor, exact on a tree
Z	a partition function; $\log Z$ the evidence
$T,\ \beta = 1/T$	a temperature and its inverse; a schedule is $T(t)$ over a budget of t steps

Two conventions are worth stating because they are load-bearing later. Optimization is always *minimization*, so a likelihood-based objective is a negative log-likelihood. And $\mathcal{L}$ is a function of $\boldsymbol{\theta}$, the unconstrained vector, not of the parameters a person names—the two are connected by ϕ and Section 2.2 explains why the distinction is not cosmetic.

A third convention governs the symbols the problem sections keep. Every model below is an instance of the factor graph of Section 2.1, and its variables are the x_i of that section; but a spin is written σ_i and a hidden state z_t, because those are the names a reader arrives with. The identification is made once, where each model is stated as a factor graph, and not repeated: $x_i \equiv \sigma_i$ on a Potts graph, $x_t \equiv z_t$ on a chain, $x_u \equiv z_u$ at a node of a tree. Every section then carries the same four parts—the model, the model as a factor graph, the algorithm as the instance of Equation 2 or of the optimization it is, and the property that pins it—so that what differs between the problems is visible as content and not as presentation.

2 The Shape All Four Problems Share

Each problem class below couples a *combinatorial* search over a structure to a *continuous* optimization of parameters given that structure. The two are not interleaved steps of one optimizer: a structural move changes what the parameter vector *means* and how long it is, so it constructs a new objective rather than taking a step inside an existing one. Everything else in this document follows from that division.

2.1 Factor Graphs: The One Structure

A tree under a substitution model, a Potts model on a graph and a hidden Markov chain are one object: a set of discrete variables $x_1, \ldots, x_N$ and a set of factors ψ_a, each a non-negative

function of a subset $x_{\partial a}$, with

$$p(x) = \frac{1}{Z} \prod_a \psi_a(x_{\partial a}), \qquad Z = \sum_x \prod_a \psi_a(x_{\partial a}). \tag{1}$$

The bipartite graph between variables and factors is the factor graph [49, 44]. In its *normal* (Forney) form every variable is an edge between exactly two factors, a variable shared by more factors passing through an equality factor $\delta(x, x', \dots)$ [28]; the form changes the drawing and not the distribution, which is checked by computing the same marginals on both. Observations enter as factors: a leaf's state is an indicator, an emission is a unary factor over the hidden state, so the graph is over latent variables only and a density and a probability are handled alike.

Sum-product sends, from factor a to variable i,

$$m_{a\to i}(x_i) = \sum_{x_{\partial a \setminus i}} \psi_a(x_{\partial a}) \prod_{j \in \partial a \setminus i} m_{j\to a}(x_j), \qquad m_{i\to a}(x_i) = \prod_{b \in \partial i \setminus a} m_{b\to i}(x_i), \tag{2}$$

and on a tree one leaf-to-root and one root-to-leaf pass are exact (Appendix A.3): on the tree's factor graph this is pruning, Equation 18; on a chain it is the forward recursion, Equation 32; on a pairwise graph it is Equation 23. On a loopy graph the fixed point is the Bethe approximation of Equation 24. Replacing the sum by a maximum gives max-product, whose max-marginals carry the MAP assignment—on a chain, Viterbi. One implementation of Equation 2 therefore serves all three problem classes, and each is pinned to the evaluator that predates it.

A fourth shape. The coupled spatio-sequential model puts a Potts prior on spatial labels $\ell_n \in \{1..M\}$, one hidden chain $k_{s,m}$ per class, and a *gated* emission joining one label to one chain state:

$$\log p(\ell, k, x) = \beta \sum_{\langle n,n' \rangle} J_{nn'}\, \delta(\ell_n, \ell_{n'}) + \sum_m \Big[\log \pi_m(k_{1,m}) + \sum_{s>1} \log A_m(k_{s-1,m}, k_{s,m}) \Big] + \sum_n \sum_s \log P(x_{sn} \mid k_{s,\ell_n}, \theta_{\ell_n} \tag{3}$$

which is Equation 1 with pairwise, chain and degree-two gated factors, and no fourth code path. Section 6 states the model in full, with the estimator its structure dictates.

A parity-check code. A low-density parity-check code is Equation 1 with binary variables, a unary factor per bit carrying the channel's log-likelihood ratio, and a hard parity constraint per row of the check matrix (Section 7). It is the problem message passing was invented for [30], and the fourth problem class: its decoder is Equation 2 specialised to one number per message, and is held to the general implementation on codes small enough for both.

What pins the general algorithm. Equation 2 is validated where each of its instances already has an oracle, and by nothing new: on a tree-shaped Potts graph against enumeration over $q^{|V|}$ configurations, on a chain against the sum over k^T paths, on a tree at one site against pruning, and on a loopy graph against the pairwise belief propagation of Equation 23—the same approximation reached by two codes, which is agreement and not truth. The Forney form is pinned by computing the same marginals on both drawings. A general algorithm that agreed only with itself would be a second opinion, not an evaluator (Section 8).

2.2 The Continuous Optimization Abstraction

Let the objective be a differentiable scalar $\mathcal{L}(\boldsymbol{\theta})$ over an unconstrained vector $\boldsymbol{\theta} \in \mathbb{R}^d$, with a map ϕ carrying $\boldsymbol{\theta}$ to the constrained parameters a model is stated in: positive branch lengths through a logarithm, a distribution over k states through a softmax on $k-1$ free values, a positive scale through a logarithm.

Why the map and not a projection. An optimizer that must be *stopped* from leaving the feasible set will eventually leave it, and a constrained parameter that reaches its boundary has no interval around it worth reporting. Constraining by construction removes both failures: every point in $\mathbb{R}^d$ is feasible, and the gradient $\nabla_{\boldsymbol{\theta}}\mathcal{L}$ is defined everywhere.

Why the map must be injective. Where a reparameterization leaves $\mathcal{L}$ unchanged, the direction along it is not estimable: the observed information is singular and an interval is undefined. A softmax on k free values has exactly this flat direction—adding a constant to every value changes nothing—which is why $k-1$ are used. Removing the freedom is the constraint map's job, not the optimizer's.

What follows for inference. Standard errors come from the observed information at the optimum, pushed through ϕ by the delta method: $\mathrm{Var}(\phi(\boldsymbol{\theta})) \approx J\Sigma J^\top$ with J the Jacobian of ϕ. This is an *asymptotic* statement, exact only in the large-sample limit, which is why Section 8 makes realized interval coverage a measurement rather than an assumption.

2.3 Sampling the Posterior, and Adapting the Sampler

Where the delta-method interval is an approximation, the posterior can be sampled instead and an interval read as a quantile. Hamiltonian Monte Carlo [60] reads $\mathcal{L}$ as a negative log density, adds a momentum $p \sim \mathcal{N}(0, M)$, and proposes by integrating the Hamiltonian $H(\boldsymbol{\theta}, p) = \mathcal{L}(\boldsymbol{\theta}) + \frac{1}{2}p^\top M^{-1}p$ with a symplectic integrator for L steps of size ε, accepting with probability $\alpha = \min\{1, \exp(-\Delta H)\}$. The integrator is reversible and volume-preserving, so ΔH is the whole of the acceptance ratio; it is exact on the flow it conserves, so ΔH is small and bounded rather than growing with L. A temperature enters as the momentum's variance, and a diagonal mass $M = \mathrm{diag}(1/\sigma_i^2)$ enters as a change of coordinates, $\phi_i = \boldsymbol{\theta}_i/\sigma_i$: the unit-mass dynamics on $\mathcal{L}(\sigma \odot \phi)$ are the mass-M dynamics on $\boldsymbol{\theta}$ term for term, so one integrator serves every metric and every temperature.

Adaptation, as a warm-up. The step ε that is right for one posterior is wrong for the next, and the coordinates' scales σ_i are the posterior's. Both are set in a warm-up whose draws are discarded, after which the chain runs at fixed values and is a Markov chain with the target as its stationary distribution; a chain that kept adapting would not be. The scales are the sample standard deviations of a warm-up window. The step is driven to a target acceptance δ by dual averaging [39, Sec. 3.2]: after proposal m with acceptance probability α_m,

$$\bar{h}_m = \Big(1 - \frac{1}{m + t_0}\Big)\bar{h}_{m-1} + \frac{\delta - \alpha_m}{m + t_0}, \qquad \log \varepsilon_m = \mu - \frac{\sqrt{m}}{\gamma}\,\bar{h}_m, \qquad \log \bar{\varepsilon}_m = m^{-\kappa}\log \varepsilon_m + (1 - m^{-\kappa})\log \bar{\varepsilon}_{m-1}, \tag{4}$$

with $\mu = \log(10\,\varepsilon_0)$ and the constants γ, t_0, κ setting the rate; the iterates ε_m do not converge, the averaged $\bar{\varepsilon}_m$ does, and the chain runs at the last of those. The same iteration is run twice, once at unit mass to estimate the scales and once on the metric they define, because a metric that has rescaled the coordinates has changed the step that is right for them.

Regime. On a target that is locally quadratic the leapfrog energy error is bounded and oscillatory, so $\alpha(\varepsilon)$ stays near one up to the stability limit $\varepsilon\,\omega_{\max} < 2$ on the stiffest direction and falls to zero past it: a target of $\delta = 0.65$ lies *on* that cliff, and an iteration driven by a single Metropolis probability per proposal, whose standard deviation is a sizeable fraction of one, oscillates across it and averages to a step whose acceptance is not δ. Two remedies, both measured rather than assumed: each proposal's step is drawn uniformly from $\bar{\varepsilon}\,(1 \pm j)$ [60, Sec. 5.4.2.2], which averages α over the band and turns the cliff into a slope with one root; and γ is set larger than the value published for a trajectory-averaged statistic, since the iterates' spread

in $\log \varepsilon$ scales as $1/(\gamma\sqrt{m})$. The chain after warm-up keeps the jitter, since that is the step its acceptance was adapted for, and every draw is still from a kernel with the right stationary distribution.

A ladder from what it measures. The same principle sets a tempering ladder for Section 4.8. What makes a ladder right is the exchange acceptance of Equation 30 between each neighbouring pair—near zero the replicas are independent chains, near one a rung is redundant—so a warm-up measures it on a candidate ladder and revises: a pair below a stated band is bisected at the geometric mean, a rung both of whose pairs are above the band is removed, and a pair above the band beside one inside it has their shared rung moved geometrically halfway toward the far end of the pair inside. Geometric, because the exchange ratio depends on the temperatures through $\beta_i - \beta_j$, so for an energy whose spread is set by the temperature the acceptance is a function of the ratio of neighbours. The endpoints are the caller's and are never moved; the warm-up's cost in sweeps is charged to any comparison against a ladder set by hand.

What pins it. Equation 4 is arithmetic and is stepped by hand against the implementation; the mass matrix is a change of coordinates and is held to a mass-matrix leapfrog written out, to round-off. What a draw costs is the effective sample size per gradient evaluation, with the size estimated by the initial positive sequence [32] and that estimator held to an autoregressive chain whose integrated autocorrelation time is a closed form. The statistics are then three: the adapted chain's acceptance lands within a stated distance of δ over independent seeds; its posterior means and spreads agree with a fixed-parameter chain's within Monte Carlo error, on the analytic Gaussian and on a four-taxon tree; and the adapted ladder's exchanges lie inside the band on the frustrated lattice, with its ground-state hit rate at equal sweeps reported beside the hand ladder's. The fixed-parameter samplers are unchanged and remain the oracles.

2.4 The Discrete Search as a Decision Process

The structural search is a Markov decision process [82]: the state is the current structure with its fitted parameters, an action is a move from the neighbourhood $\mathcal{N}(\cdot)$, and the reward is the improvement in the objective. An episode ends on a step budget or when no move improves the score—a property of the state, and the same stopping rule the greedy baseline obeys, which is what makes the two comparable. Because the reward is a difference, the undiscounted return telescopes,

$$G = \sum_{t=0}^{T-1} R_t = \sum_{t=0}^{T-1} \big[\mathcal{L}(s_t) - \mathcal{L}(s_{t+1})\big] = \mathcal{L}(s_0) - \mathcal{L}(s_T), \tag{5}$$

the improvement between the first and last state whatever path joined them. That is what licenses an undiscounted return—a discount would prefer improvement found early over the same improvement found late—and it is why a truncation landing between a sacrifice and its payoff teaches the opposite of the truth, so a truncated episode is recorded as one.

The policy is a softmax over the scores of the *available* actions, $\pi(a \mid s) \propto \exp f(s,a)$ for $a \in \mathcal{N}(s)$, so a neighbourhood of any size is admissible. A scorer with a single weight on the improvement a move buys is an inverse temperature, and greedy search is its zero-temperature limit; what such an agent can learn is exactly how much it should accept a worsening move to reach a better optimum, which is the credit-assignment question this search makes sharp. It is trained by the score-function estimator with a baseline,

$$\nabla J = \mathbb{E}\Big[\sum_t \nabla \log \pi(a_t \mid s_t)\,\big(G_t - b(s_t)\big)\Big], \qquad G_t = \sum_{u \geq t} R_u, \tag{6}$$

where a baseline b independent of the sample it is subtracted from leaves the estimator unbiased and is there to reduce its variance, not its bias: G_t is a sum of objective differences whose scale

is the problem's, so an uncentred gradient varies by orders of magnitude between instances. The comparison against the greedy baseline is at a matched budget of objective evaluations per decision, never in wall-clock time.

What pins it. Equation 5 is an identity and is asserted exactly on every recorded episode. Equation 6 is an expectation, so its estimator is pinned against the gradient of the exact expected return, which is computable by enumerating every episode on a small environment, and the baseline's unbiasedness is asserted by the same enumeration: the expected gradient with and without b agree. An environment small enough to enumerate is therefore the oracle for the learning machinery, exactly as a tree small enough to enumerate is the oracle for the search.

2.5 Actor–Critic and Proximal Policy Optimization

The baseline in (6) may depend on the state. The natural choice is the state value $V^\pi(s) = \mathbb{E}[G_t \mid s_t = s]$, the improvement the search still expects from s, estimated by a *critic* fitted to sampled returns; the estimator is then weighted by the *advantage* $A_t = G_t - V(s_t)$, unbiased for any critic and of least variance in this family when the critic is exact (Appendix B). Where the return is exact by enumeration, so is V^π, and a critic is scored against it rather than against its own loss. The Monte Carlo return and the one-step bootstrap $R_t + V(s_{t+1})$ are the two ends of the *generalized advantage* [78], at $\gamma = 1$,

$$A_t^{(\lambda)} = \sum_{l \geq 0} \lambda^l \delta_{t+l}, \qquad \delta_t = R_t + V(s_{t+1}) - V(s_t), \tag{7}$$

with $V(s_T) = 0$ at a terminal state and the critic's value at a truncated one; $\lambda = 1$ recovers $G_t - V(s_t)$ and $\lambda = 0$ the temporal difference, trading the critic's bias against the return's variance.

The score-function estimator is valid only for the policy that collected the episodes. *Proximal policy optimization* [79] reuses a batch for several gradient steps by importance-weighting each decision with $\rho_t = \pi(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$ and clipping the ratio's effect on the objective,

$$\mathcal{L}_{\text{PPO}} = \mathbb{E}\Big[\sum_t \min\big(\rho_t A_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\, A_t\big)\Big], \tag{8}$$

so that an update cannot move the policy far from the one the advantages were estimated under. At the collecting policy every ratio is one and, without clipping, the gradient of (8) is the actor–critic's; the regression suite pins that identity, and measures on the Potts chain what the reuse buys: the fraction of the 81 starts from which the trained policy reaches the enumerated optimum, at a matched number of episodes. The same ratio makes the update valid when a *behaviour* policy collected the episodes: with π_{old} replaced by β, the ϵ-greedy wrapper of the policy being trained, on an ϵ schedule, the search can leave a local optimum the softmax would settle into, and the estimator is corrected by π/β.

2.6 Planning with a Learned Prior and Value

A policy answers "which move" from what it has seen; a planner looks ahead. From the current state, each of n simulations descends a tree of successors choosing, at every expanded node, the child that maximizes

$$Q(s,a) + c\,\pi(a \mid s)\,\frac{\sqrt{N(s)}}{1 + N(s,a)}, \tag{9}$$

the mean return backed up through that child plus an exploration bonus that is the prior the policy gives the move, decaying with the child's visits [80]; the first unexpanded child is scored by the critic and the return backed up to the root. The root's visit counts, normalized, are a policy

that improves on the prior, and *expert iteration* [6] trains the policy toward that distribution by cross-entropy and the critic toward the achieved return, so the planner's answers become the policy's. Every expansion evaluates a successor, so the search is counted in the unit the rest of this document counts, and the comparison with greedy states the rate and the evaluations together (Appendix B).

2.7 Surrogates and Bounds

The evaluation a discrete search pays for—a fit of $\boldsymbol{\tau}$ per topology, or $\log Z$ for a lattice—is what makes the search expensive, so each problem carries *surrogates*: cheaper functions of the structure that stand in for it. A surrogate makes a claim, a lower bound, an upper bound, or a point prediction, and the claim is part of the object. An analytic bound is proved (Appendix C) and *certified*: checked against the exact value on every structure an oracle can score, with no violation allowed. A learned predictor is *calibrated* instead, shifted by a quantile of its held-out residuals, and certified to the coverage it claims. The search ranks a neighbourhood by a surrogate and fits only the top candidates; the accepted move is always evaluated in full, so a surrogate changes what is fitted and never what is reported.

For a topology the maximized log-likelihood $\ell^*(\mathcal{T}) = \max_{\boldsymbol{\tau}} \ell(\mathcal{T}, \boldsymbol{\tau})$ is bracketed by two one-pass quantities. Any feasible lengths give a lower bound, and lengths $\hat{\boldsymbol{\tau}}$ solved by non-negative least squares from the pairwise Jukes–Cantor distances $\hat{d}_{ab}$ are feasible:

$$\ell(\mathcal{T}, \hat{\boldsymbol{\tau}}) \;\le\; \ell^*(\mathcal{T}), \qquad \hat{\boldsymbol{\tau}} = \arg\min_{\boldsymbol{\tau} \ge 0} \sum_{a<b} \Big(\sum_{e \in \mathrm{path}(a,b)} \tau_e - \hat{d}_{ab} \Big)^2 . \tag{10}$$

Under Jukes–Cantor $\mathbf{P}(t) = a\,\mathbf{I} + (1-a)\,\mathbf{J}/k$ with $a = e^{-kt/(k-1)}$, so a site likelihood is multilinear in the a_e and its maximum over the box $[0,1]^{|E|}$ sits at a vertex, where every branch either forbids a change or forgets the parent's state at cost $1/k$. With the tree rooted at the first leaf, the vertex maximum of site s is $\pi(x_{1,s})\, k^{-F_s(\mathcal{T})}$ for the site's Fitch score F_s, whence

$$\ell^*(\mathcal{T}) \;\le\; \sum_s \log \pi(x_{1,s}) \;-\; F(\mathcal{T}) \, \log k. \tag{11}$$

Both apply to a subtree as they do to the tree.

For a lattice the partition function is bracketed by two variational quantities. The naive mean-field bound is Gibbs' inequality at a product distribution $q = \prod_i q_i$, tightest at its fixed point:

$$\log Z \;\ge\; \sum_i \sum_\sigma q_i(\sigma)\, h_i(\sigma) + \sum_{\langle i,j \rangle} J_{ij} \sum_\sigma q_i(\sigma)\, q_j(\sigma) - \sum_i \sum_\sigma q_i(\sigma) \log q_i(\sigma). \tag{12}$$

Because $\log Z$ is convex in the parameters, Jensen's inequality over a distribution ρ on spanning trees T, with parameters $\theta(T)$ supported on T and averaging to θ, gives an upper bound each term of which is exact by one leaf-to-root pass:

$$\log Z(\theta) \;\le\; \sum_T \rho(T) \, \log Z\big(\theta(T)\big), \qquad \sum_T \rho(T)\, \theta(T) = \theta, \tag{13}$$

the tree-reweighted bound of Wainwright et al. [87]. From any bracket $L \le \log Z(\beta) \le U$ at inverse temperature β, and $e^{-\beta E_{\min}} \le Z(\beta) \le k^N e^{-\beta E_{\min}}$,

$$-\frac{U}{\beta} \;\le\; E_{\min} \;\le\; \frac{N \log k - L}{\beta}. \tag{14}$$

3 Problem Statement: Phylogenetic Inference

3.1 The model

Character evolution along a branch is a continuous-time Markov chain with rate matrix $\mathbf{Q}$; the transition probabilities over a branch of length t are the matrix exponential $\mathbf{P}(t) = \exp(\mathbf{Q}t)$ [26]. A time-reversible model factors as $\mathbf{Q} = \mathbf{S}\,\mathrm{diag}(\boldsymbol{\pi})$ with $\mathbf{S}$ symmetric, and then satisfies detailed balance, $\pi_i \mathbf{Q}_{ij} = \pi_j \mathbf{Q}_{ji}$.

The simplest such model is Jukes–Cantor on k states: every substitution equally likely and $\boldsymbol{\pi}$ uniform, with $\mathbf{Q}$ scaled so that one unit of branch length is one expected substitution per site,

$$-\sum_{i \in \mathcal{A}} \pi_i \, \mathbf{Q}_{ii} = 1, \tag{15}$$

which puts the transition probabilities in closed form [26, 22]:

$$\mathbf{P}_{ij}(t) = \frac{1}{k} + \Big(\delta_{ij} - \frac{1}{k}\Big) \, \exp\Big(-\frac{k}{k-1}\, t\Big). \tag{16}$$

Rows sum to one, $\mathbf{P}(0) = \mathbf{I}$, and every entry tends to $1/k$ as $t \to \infty$; the general time-reversible model replaces the single rate by $\mathbf{S}$ and $\boldsymbol{\pi}$ and keeps Equation 15.

Given a topology $\mathcal{T}$ over n taxa, branch lengths $\boldsymbol{\tau}$, and an alignment $\mathbf{D}$ of L sites, sites evolve independently and the likelihood marginalizes every assignment z of states to the internal nodes:

$$p(\mathbf{D} \mid \mathcal{T}, \boldsymbol{\tau}, \mathbf{Q}) = \prod_{s=1}^{L} \sum_{z} \pi_{z_\rho} \prod_{(u \to v)} \mathbf{P}_{z_u z_v}(\tau_v), \tag{17}$$

the product over the branches $(u \to v)$ of the rooted tree with the leaves fixed by the data—$k^{\,n-2}$ terms per site for a binary tree. Summing it directly is the oracle every faster evaluation of it is checked against.

3.2 As a factor graph

One site is one instance of Equation 1. The variables are the states $x_u \equiv z_u \in \mathcal{A}$ at every node u of the rooted tree, leaves included. The factors are the root prior $\psi_\rho(x_\rho) = \pi_{x_\rho}$, one pairwise factor per branch, $\psi_v(x_u, x_v) = \mathbf{P}_{x_u x_v}(\tau_v)$ for u the parent of v, and one indicator per leaf, $\psi_\ell(x_\ell) = \mathbb{1}[x_\ell = \text{observed}]$, which folds the data in. The bipartite graph is a tree, so Equation 2 is exact, and Z is the site likelihood $p(\mathbf{D}_s \mid \mathcal{T}, \boldsymbol{\tau}, \mathbf{Q})$; sites being independent (Equation 17), the alignment's likelihood is the product of L such Z. In the Forney form every internal node's state is an edge through an equality factor of degree three—two branches and, at the root, the prior—which is the drawing Section 6 uses.

3.3 The algorithm: simulation

The generative direction of Equation 17 is a pre-order walk: draw the root state, then each node's state from the row of its parent's state. One site is Algorithm 1; L sites are L independent runs, and the empirical substitution frequencies along any branch converge to Equation 16, which is the check a simulator is held to—against the closed form, and never against the likelihood that will later be fitted to its output.

3.4 The algorithm: pruning

Felsenstein's pruning evaluates that marginal in $O(nLk^2)$ rather than $O(Lk^{n-2})$, by a post-order recursion carrying a partial likelihood $L_u(i)$—the probability of the data below node u given

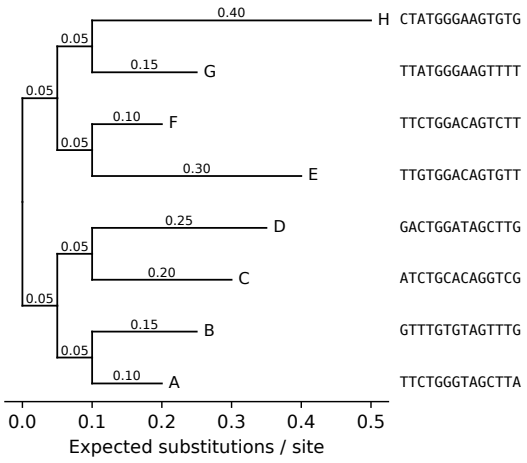

Figure 1: Simulated topology for 8 taxa under the Jukes-Cantor model (seed 20260904), branch lengths labelled in expected substitutions per site. The first 14 simulated sites are shown beside each leaf, in tree order, so the alignment the likelihood consumes can be read off against the topology that generated it.

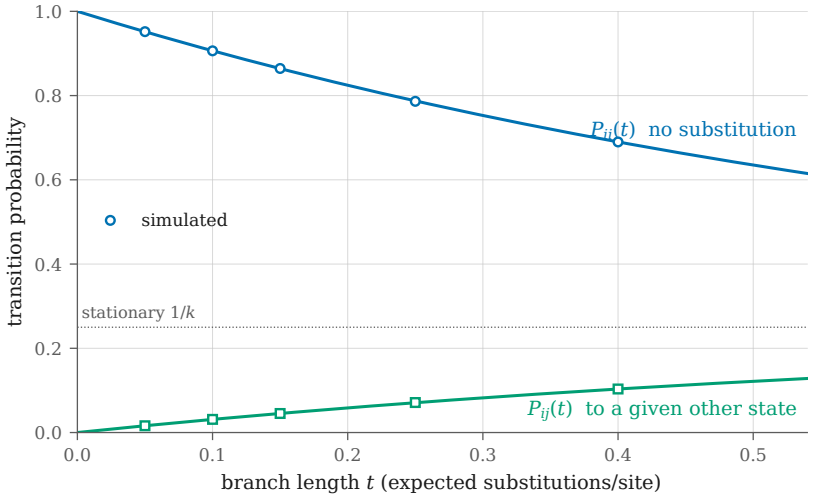

Figure 2: Closed-form Jukes-Cantor transition probabilities for k = 4 (curves) against the frequencies the simulator produced (markers), one pair per branch of the 5-branch fixture tree. Simulated at seed 20260902 over 200000 sites. Largest deviation 1.07e-03, within the fixture's declared Monte Carlo tolerance of 1e-02. The simulator is validated against this analytic form, never against the likelihood code it feeds.

Algorithm 1 Forward simulation of one site under $(\mathcal{T}, \boldsymbol{\tau}, \mathbf{Q})$

draw $z_\rho \sim \boldsymbol{\pi}$
for each non-root node v in pre-order, with parent u **do**
    draw $z_v \sim \mathbf{P}_{z_u,\cdot}(\tau_v)$
end for
return the leaf states $(z_\ell)_{\ell \in \text{leaves}(\mathcal{T})}$

that u is in state i:

$$L_u(i) = \prod_{v \in \text{children}(u)} \sum_{j \in \mathcal{A}} \mathbf{P}_{ij}(t_v)\, L_v(j), \tag{18}$$

with $L_u(i) = \mathbb{1}[\,u \text{ observed in state } i\,]$ at a leaf, and the site likelihood read off at the root,

$$p(\mathbf{D}_s \mid \mathcal{T}, \boldsymbol{\tau}, \mathbf{Q}) = \sum_{i \in \mathcal{A}} \pi_i\, L_\rho(i) \tag{19}$$

[26, 22]; Appendix A.1 derives the recursion from the marginalization. Products of many small numbers underflow, so each node's partials are rescaled and the logarithm of the scaling accumulated separately; the rescaling must remain differentiable, since $\boldsymbol{\tau}$ and $\mathbf{Q}$ are what the continuous optimization fits.

Equation 18 is Equation 2 run leaf to root with the root as the last variable: $L_u(i)$ is the product over u's children of the factor-to-variable messages $m_{\psi_v \to u}(i)$, each of which sums the branch factor against the message arriving from below, and Equation 19 is the final factor-to-variable message from the prior. Replacing the sum by a maximum gives the most probable joint assignment of ancestral states; in the limit of short branches its cost counts substitutions, which is Fitch parsimony [26], the second criterion the search is scored against.

Both parsimony scores are the same post-order pass with no branch lengths. Fitch's recursion [27] carries at each node v the set S_v of states reachable at the minimum cost so far, and a count F of changes:

$$S_v = \begin{cases} S_a \cap S_b & S_a \cap S_b \neq \emptyset, \\ S_a \cup S_b \quad \text{and} \quad F \leftarrow F + 1 & \text{otherwise}, \end{cases} \tag{20}$$

for the children a, b of v, folded pairwise at a trifurcation, a leaf's set being its observed state. Sankoff's recursion [74] replaces the set by a cost vector under a step matrix $W \in \mathbb{R}^{k \times k}_{\geq 0}$, W_{ij} the cost of a change from a parent in state i to a child in state j:

$$S_v(i) = \sum_{c \in \text{ch}(v)} \min_j \big[\, W_{ij} + S_c(j) \,\big], \qquad S_{\text{leaf}}(i) = \begin{cases} 0 & i = x_{\text{leaf}}, \\ \infty & \text{otherwise}, \end{cases} \tag{21}$$

and a site's score is $\min_i S_{\text{root}}(i)$. It is Equation 18 in the min-plus semiring, and under the unit matrix $W = \mathbf{J} - \mathbf{I}$ it returns Fitch's count exactly. The score is that of a rooted tree: an asymmetric W scores each rooting of one unrooted topology differently, and a search over unrooted topologies is well defined only when W is a metric, where every rooting agrees.

Regime. Exact on every input, because a tree has no cycle. The one failure mode is numerical—underflow of a product of n factors each below one—and it is removed by the rescaling, not tolerated.

What pins it. Direct marginalization, Equation 17, at the sizes it allows, to a relative tolerance (Section 8.5). Two invariants past those sizes: the pulley principle, that a reversible model's likelihood does not depend on where the root is placed along a branch, and a gradient against central differences relative to its own norm. And the recovery test of Section 8: on simulated data the fitted branch lengths cover the truth at their nominal rate, and the search recovers the generating topology at a stated site count.

3.5 The algorithm: discrete search

The number of unrooted binary topologies on n taxa is $(2n-5)!!$ [73], so exhaustive enumeration is an oracle at small n and nothing more. Two neighbourhoods are standard. *Nearest-neighbour interchange* (NNI) swaps the subtrees adjacent to one internal edge, giving $|\mathcal{N}_{\text{NNI}}(\mathcal{T})| = 2(n-3)$.

Subtree prune and regraft (SPR) detaches a subtree and reattaches it at another edge, giving $2(n-3)(2n-7)$ and containing NNI. Neither is complete in one step; both are complete in the transitive closure, which for NNI is the connectivity of the associated graph [26] and is inherited by SPR because it dominates NNI.

Hill climbing over either neighbourhood fits the continuous parameters of every candidate and moves to the best; its budget is counted in candidates scored, never in seconds, so a run reproduces from its seed. The same climb with Equation 20 or 21 in place of the fit is *large parsimony* [26]: nothing continuous is fitted, a candidate costs one pass, and the budget and the oracle are the same. A candidate that shares all but a few branches with its parent may start its fit from the parent's lengths, a branch being identified by the leaf bipartition below it rather than by a node's name; and a neighbourhood may be *ranked* by one evaluation at those lengths before the few best are fitted, the accepted move always in full, so every reported score is an optimum. Whether the ranking finds the fitted best is a property of the move set and is measured per move set, not assumed.

What pins it. Enumeration of the $(2n-5)!!$ topologies referees the search below eight taxa: the climb must reach the enumerated maximum, and the neighbourhood counts above are asserted exactly over every topology. A warm start is pinned by reaching the cold fit's optimum; a cached partial likelihood by equalling the recomputed one bitwise, since it is the same arithmetic in the same order. Large parsimony is held to the same enumeration from every start, and on the Felsenstein-zone fixture its enumerated optimum is asserted to be the wrong tree while the likelihood optimum is the right one, because there the criterion's failure is the theorem of Section 8 and not a defect of the search.

4 Problem Statement: Potts Models in an External Field

4.1 The model

On a graph $\mathcal{G}$, each node i carries a spin $\sigma_i \in \{1, \ldots, q\}$ and the energy is

$$H(\sigma) = -\sum_{\langle i,j \rangle} J_{ij}\, \delta(\sigma_i, \sigma_j) - \sum_i h_i(\sigma_i), \tag{22}$$

with couplings J_{ij} and an external field h_i [56]. The Boltzmann distribution is $p(\sigma) \propto \exp(-H(\sigma))$, and its normalizer sums over $q^{|V|}$ configurations.

4.2 As a factor graph

The variables are the spins, $x_i \equiv \sigma_i$. The factors are one unary $\psi_i(\sigma_i) = e^{h_i(\sigma_i)}$ per node and one pairwise $\psi_{ij}(\sigma_i, \sigma_j) = e^{J_{ij}\,\delta(\sigma_i,\sigma_j)}$ per edge, so Equation 1 is the Boltzmann distribution and Z its normalizer. At a temperature T the factors are raised to the power $\beta = 1/T$, which is the same graph with J and h scaled; nothing else about the model changes, and that is why a tempered chain can be held to the unscaled model's enumeration. The graph is a tree exactly when $\mathcal{G}$ is, which is the regime every exact claim below is confined to.

4.3 Why an odd cycle is the whole story

A two-state antiferromagnet ($J < 0$) wants every edge to disagree, which is possible exactly when $\mathcal{G}$ is 2-colourable. So every bipartite lattice—a chain, a square lattice—has a trivial, unfrustrated ground state and says nothing about a hard problem. A geometry containing odd cycles is what makes the ground state non-trivial, and where a counting argument closes exactly it fixes that ground state at every size. This is the difference between a fixture that exercises an optimizer and one that does not.

4.4 The algorithm: belief propagation

Exact marginals come from enumeration at small sizes and from belief propagation on a tree, where it is exact; on a loopy graph it is an approximation whose accuracy is a property of the graph rather than of the implementation [44, 29]. It is Equation 2 with every factor pairwise or unary: composing the variable-to-factor message with the unary factor gives the node-to-node form, in which the message from i to a neighbour j is

$$m_{i\to j}(\sigma_j) \;\propto\; \sum_{\sigma_i} \exp\big(J_{ij}\,\delta(\sigma_i,\sigma_j) + h_i(\sigma_i)\big) \prod_{l\in\partial i\setminus j} m_{l\to i}(\sigma_i), \tag{23}$$

iterated to a fixed point, with beliefs $b_i(\sigma_i) \propto e^{h_i(\sigma_i)} \prod_{l\in\partial i} m_{l\to i}(\sigma_i)$ at a node and likewise on an edge. The messages are carried in the logarithm and normalized every sweep, because a coupling of a few units on a small lattice already puts a product of exponentials past what a linear recursion holds. The Bethe free energy of the fixed point,

$$F_{\mathrm{Bethe}} = \sum_{\langle i,j\rangle} \sum_{\sigma_i,\sigma_j} b_{ij}\,\big(E_{ij} + \log b_{ij}\big) + \sum_i \sum_{\sigma_i} b_i\,E_i - \sum_i (d_i - 1) \sum_{\sigma_i} b_i \log b_i, \tag{24}$$

with $E_{ij} = -J_{ij}\,\delta(\sigma_i,\sigma_j)$, $E_i = -h_i(\sigma_i)$ and d_i the degree of i, equals $-\log Z$ exactly on a tree and approximates it on a loopy graph [92, 56]; Appendix A.4 states why, and Appendix A.3 places the tree case. A fixed point the iteration never reached is not an estimate of anything, so non-convergence is a refusal rather than a number.

Regime. Exact on a tree, in two passes. On a loopy graph the flooding schedule is iterated with damping to a tolerance and returns the Bethe approximation, and the tree schedule is refused rather than run, because a number that looks exact and is not is the worst output an evaluator can produce.

What pins it. On a tree, $\log Z$ and every marginal against enumeration to a stated relative tolerance. On a loopy graph nothing asserts agreement with the truth—that would assert something false—and what is asserted is the structure the physics requires: exact at zero coupling, and a deviation from the exact transfer matrix that grows away from it and is reported as a measurement.

4.5 The algorithm: ground states as cuts

With two states write $\sigma_i \in \{-1, +1\}$. The pairwise term of Equation 22 is constant on agreeing edges and pays J_{ij} on disagreeing ones, so minimizing H minimizes the weight of the *cut*—the set of edges whose endpoints differ—plus the field:

$$H(\sigma) = \sum_{\langle i,j\rangle:\sigma_i\neq\sigma_j} J_{ij} - \sum_i h_i(\sigma_i) + \text{const.} \tag{25}$$

When every $J_{ij} \geq 0$ the pairwise term is submodular, the field becomes a terminal edge to a source or a sink, and the ground state is a minimum s–t cut, exact by max-flow in polynomial time [36, 20]. When couplings are negative the problem is maximum cut, which is NP-hard; the semidefinite relaxation rounded by a random hyperplane gives a cut whose expected weight is at least $\alpha_{\mathrm{GW}} = 0.87856$ of the optimum,

$$\mathbb{E}[\,w(\mathrm{cut})\,] \;\geq\; \alpha_{\mathrm{GW}}\;\max_{\sigma} w(\sigma), \qquad \alpha_{\mathrm{GW}} = \min_{0<\vartheta\leq\pi} \frac{2}{\pi}\,\frac{\vartheta}{1-\cos\vartheta}, \tag{26}$$

and the relaxation's own value bounds the optimum from above [33]. The bound is the claim that holds at every size; the realized ratio on an instance is measured beside it and is never quoted as the guarantee.

Regime. Exact where the couplings are submodular and refused rather than approximated where they are not (Section 8): a wrong ground state on a structured instance is indistinguishable from a right one at any size worth solving. The relaxation runs on any instance and certifies a ratio, not an optimum.

What pins it. Enumeration over $2^{|V|}$ configurations where it reaches; where it does not, a *reduction*: on a bipartite graph the maximum cut is every edge, on a uniform ferromagnet the ground state is constant, and a construction error in the cut graph yields a labelling that is merely worse, which only a regime where the general construction must reproduce a simpler validated one will catch. A fixture whose answer is trivial measures nothing, so the odd cycles of Section 4 are what the instances carry.

4.6 The algorithm: alpha expansion

With $q > 2$ states the ground state is NP-hard in general, and alpha expansion approximates it by a sequence of exact two-state problems [17]. An *expansion* on label α lets every site keep its label or switch to α; the best such move is the minimum cut of a two-state graph built from the current labelling, exact when the pairwise term is a metric on labels, which $\delta(\sigma_i, \sigma_j)$ is. Cycling over α until no expansion lowers the energy reaches a local minimum within a constant of the global one,

$$H(\hat{\sigma}) \;\le\; 2\,c\,H(\sigma^\star), \qquad c = \max_{\alpha\neq\beta'} d(\alpha,\beta') \Big/ \min_{\alpha\neq\beta'} d(\alpha,\beta'), \tag{27}$$

so a factor of two for the Potts metric, where $c = 1$. The guarantee assumes each expansion is solved to optimality; where it is solved approximately the certificate is optimistic, and what can be asserted is the symptom—a value an exact solve could not have produced.

What pins it. The two-label case must reproduce the exact minimum cut of Section 4.5 exactly; enumeration referees the multi-label case where it reaches; and the realized ratio against the enumerated optimum is measured, since the bound is a theorem and the ratio is a number.

4.7 The algorithm: sampling

The single-site heat bath draws each spin from its conditional,

$$p(\sigma_i \mid \sigma_{\partial i}) \;\propto\; \exp\Big(\beta\big[\textstyle\sum_{j\in\partial i} J_{ij}\,\delta(\sigma_i,\sigma_j) + h_i(\sigma_i)\big]\Big), \tag{28}$$

one site at a time; a *sweep* is one update of every site. Near a critical coupling correlated regions span the lattice and single-site moves decorrelate slowly. Cluster moves flip such regions together [85, 90, 61]: in the Edwards–Sokal representation [24] a bond is placed between equal neighbours with probability $1 - e^{-\beta J_{ij}}$, and each bond-connected cluster is recoloured uniformly—every cluster at once in Swendsen–Wang, one cluster grown from a random seed in Wolff. The bond construction is exact at zero field only. In a field the recolouring is accepted with the Metropolis probability on the field energy alone,

$$a = \min\Big\{1,\; \exp\Big(\beta \textstyle\sum_{i\in\mathcal{C}} \big[h_i(\nu) - h_i(\mu)\big]\Big)\Big\}, \tag{29}$$

for a cluster $\mathcal{C}$ recoloured from μ to ν, which restores detailed balance (Appendix A.5).

Regime. Every move preserves the Boltzmann distribution at β, so a sweep may not stop on a state-dependent condition: composing a *number* of steps chosen from the outcome biases toward the states that triggered the stop. The bond probability is a probability only for $J_{ij} \ge 0$, and an antiferromagnet has no like-spin clusters to flip, so the cluster moves refuse a negative coupling rather than run badly.

What pins it. A sampler is validated by the distribution it converges to and never by inspection: at an enumerable size the exact Boltzmann distribution is available, and a chain is held to it by a goodness-of-fit test at a declared significance, thinned by several autocorrelation times because successive sweeps are not independent draws. The test's power is itself pinned—a move set with the field accept step removed runs, mixes, and is rejected.

4.8 Temperature, annealing and tempering

A temperature T scales the model, H/T, so a chain at T targets $\exp(-H/T)$; a *schedule* $T(t)$ over a budget of t steps is one object, shared by every consumer of it, with both endpoints reached exactly at the declared steps and a step past the end refused. Simulated annealing is the sampler on a decreasing schedule, returning the lowest-energy state seen [43]; at $T \to 0$ the heat bath is the argmin over each site's conditional, which is iterated conditional modes, so annealing and greedy descent are one search separated by the schedule alone.

Parallel tempering runs R replicas at fixed temperatures $T_1 < \cdots < T_R$ and, after each sweep, proposes to exchange the configurations of adjacent replicas, accepting with

$$a_{ij} = \min\big\{1,\ \exp\big[(\beta_i - \beta_j)(E_i - E_j)\big]\big\}, \tag{30}$$

the ratio of the joint target, a product of tempered marginals [84, 31, 23]; Appendix A.6 derives it. The hot replicas cross barriers the cold one cannot, and an exchange carries what they find down the ladder. Each replica draws from its own generator, spawned from one parent that then draws only the exchange uniforms, so no two replicas share a stream and one seed reproduces the run.

The replica at $T_1 = 1$ targets the unscaled model, so the fraction of its N recorded sweeps spent at a structure x estimates that structure's weight over the whole space,

$$\hat{w}(x) = \frac{1}{N}\sum_{n=1}^{N}\big[\sigma_1^{(n)} = x\big] \ \to\ \frac{\exp(-H(x))}{Z}, \tag{31}$$

where a weight over a neighbourhood is conditional on that neighbourhood and enumeration is refused past its cap. The structure may be a labelling, a hidden path, or a topology with H the negative fitted log-likelihood, in which case the limit is the flat-prior weight over maximized likelihoods and not a marginal over branch lengths. It is a Monte Carlo estimate, and is named a posterior weight only beside the exchange acceptance of every adjacent pair and the autocorrelation time of the indicator, which say whether the ensemble mixed.

What pins it. Every replica's marginal must still be $\exp(-H/T_r)$, enumerated from the *unscaled* model, with exchanges on: a wrong exchange ratio contaminates the cold replica with hot configurations and only that test says so. Its power is pinned by the paired negative—an exchange that omits the energy term still runs and still mixes, and is rejected on every replica. Whether tempering or annealing beats restarts of descent at an equal budget of sweeps is not a property of the algorithms but of the instance, and is measured on the frustrated, past-enumeration instances the roadmap needs, never asserted in general.

5 Problem Statement: Hidden Markov Models

5.1 The model

A hidden path $\mathbf{Z} = (z_1, \ldots, z_T)$ over k states evolves by an initial distribution $\boldsymbol{\pi}$ and a transition matrix $\mathbf{A}$; each position emits an observation from an emission family $\mathcal{E}$ indexed by its state. The evidence marginalizes k^T paths.

5.2 As a factor graph

The variables are the hidden states, $x_t \equiv z_t$. The factors are $\psi_1(z_1) = \pi_{z_1}\,\mathcal{E}_{z_1}(y_1)$ at the first position and, for $t > 1$, one pairwise $\psi_t(z_{t-1}, z_t) = \mathbf{A}_{z_{t-1}z_t}$ per transition and one unary $\mathcal{E}_{z_t}(y_t)$ per emission: the observation y_t is folded into the table, so a probability and a density enter the same way and the graph is over latent variables only. The chain is a tree, and Z is the evidence $p(\mathbf{D})$. An interior state sits on three factors, so in the Forney form it passes through an equality node—the drawing Section 6 uses for each class's chain.

The emission family is the only part that knows what an observation is. Everything else—the recursions, the expectation step, path enumeration—needs three things from it: draw an observation given a state, score one, and re-estimate itself from posterior weights. Separating it is what lets one set of recursions serve a finite alphabet, a real observation, and a count.

A density is not a probability. For a family over a countable alphabet the evidence is a probability, so $\log p(\mathbf{D}) \le 0$. For a family over the reals it is a *density* and carries no such bound. A check written against the first is false under the second, so what may be asserted is keyed on the support the model declares.

5.3 The algorithm: forward–backward

The forward recursion is Equation 18 on a chain:

$$\alpha_t(i) = \Big[\sum_j \alpha_{t-1}(j)\,\mathbf{A}_{ji}\Big]\,\mathcal{E}_i(y_t), \qquad p(\mathbf{D}) = \sum_i \alpha_T(i). \tag{32}$$

Pruning on a caterpillar tree carrying one observed leaf per internal node *is* this recursion [22, 44]: the three problem classes are one marginalization over three structures, not three unrelated models sharing an optimizer; Appendix A.2 derives both passes from it. In the terms of Equation 2, α_t is the message arriving at z_t from the left multiplied by the emission factor, the backward recursion β_t is the same pass from the right, and the posterior at a position is their normalized product,

$$p(z_t = i \mid \mathbf{D}) = \frac{\alpha_t(i)\,\beta_t(i)}{p(\mathbf{D})}, \qquad \beta_t(i) = \sum_j \mathbf{A}_{ij}\,\mathcal{E}_j(y_{t+1})\,\beta_{t+1}(j), \tag{33}$$

which is the belief b_t of Section 1. Max-product on the same chain is Viterbi,

$$\delta_t(i) = \Big[\max_j\ \delta_{t-1}(j)\,\mathbf{A}_{ji}\Big]\,\mathcal{E}_i(y_t), \tag{34}$$

with the path read back from the argmax at each step; on a chain with a unique maximum the max-marginals' argmax at every position is that path [22].

Regime. Exact for every length, the chain being a tree. The one hazard is the same underflow pruning meets, and the same remedy applies: the recursion runs in the logarithm.

What pins it. The sum over every one of the k^T paths, written out term by term with no recursion, referees the evidence, the posteriors and the Viterbi path at the lengths it allows; a gradient against central differences past them. The enumeration is the oracle two decoders are separated by, so it cannot be validated by a decoder, and it is pinned in turn against algebra: quantities that can be worked out by hand on a two-state, two-step chain.

5.4 The algorithm: expectation-maximization

Baum–Welch alternates a posterior over hidden states given the parameters with a re-estimate of the parameters given that posterior, and increases the likelihood at every iteration [53]. The initial distribution and the transitions re-estimate in closed form for every family; the emission re-estimate is the family's own, and *whether it is a formula* is the property that separates the families. A weighted mean is; a dispersion parameter whose score involves the digamma function is not, and its M step is itself an optimization—which is the case that says whether an interface built around closed forms was built correctly.

What pins it. Monotonicity, asserted at every iterate; agreement of the fixed point with a gradient fit of the same likelihood; and recovery of simulated parameters up to the permutation of hidden states, with the alignment part of the test (Section 8). Where the family's M step is itself an optimization, its refusals at the boundary of the parameter space are pinned too: a dispersion the data cannot identify is refused, not clamped.

5.5 Two decodings, and they are different answers

Viterbi returns the single path of highest joint probability; posterior decoding returns, at each position independently, the state of highest marginal probability. Where they disagree, the posterior-decoded sequence can be a path the model assigns zero probability to, because nothing constrains consecutive choices to be compatible. They agree on almost every instance, which is exactly why an implementation that computes one and reports the other survives a test suite with no case where they diverge. Equation 34 and the argmax of Equation 33 are those two decodings; the fixture on which they differ is constructed and kept, because it is the only kind of case that can tell an implementation of one from an implementation of the other.

6 Problem Statement: The Coupled Spatio-Sequential Model

6.1 The model

Observations x_{sn} are indexed by a spatial node n of a graph $\mathcal{G}$ and a sequential position $s = 1, \dots, S$. Each node carries a class label $\ell_n \in \{1, \dots, M\}$; each class m carries its own hidden chain $k_{s,m} \in \{1, \dots, K\}$; and the observation at (s, n) is emitted by the chain of the class the node belongs to, under that class's emission parameters θ_m. The joint is Equation 3. Its three terms are the three problem classes above: a Potts prior on the labels,

$$\log p(\ell) = \beta \sum_{\langle n,n' \rangle} J_{nn'}\, \delta(\ell_n, \ell_{n'}) - \log Z_{\text{Potts}}, \qquad J_{nn'} \geq 0, \tag{35}$$

ferromagnetic so that neighbouring nodes prefer one class—the penalty on disagreement, $-\beta \sum J_{nn'} \mathbb{1}[\ell_n \neq \ell_{n'}]$, is the same distribution up to a constant; one hidden Markov chain per class, in the simplest case with a circulant transition,

$$\mathbf{A}_m(i, j) = t\, \delta_{ij} + \frac{1 - t}{K - 1}\,(1 - \delta_{ij}), \tag{36}$$

whose self-transition rate t and initial distribution Π_m are fitted; and a *gated* emission, $\mathbb{1}[\ell_n = m] \log p(x_{sn} \mid k_{s,m}, \theta_m)$, which is what couples the two. Marginalizing k recovers a Potts model in a field the chains define; marginalizing ℓ recovers a mixture of hidden Markov models. Neither marginal is tractable on its own, and the structure of the joint is what dictates the estimator.

6.2 As a factor graph

The variables are the N labels and the SM chain states. The factors are the pairwise Potts factors of Section 4 on the labels, the chain factors of Section 5 on each class's states, and one gated emission factor per (n, s, m) over the pair $(\ell_n, k_{s,m})$, equal to $p(x_{sn} \mid k_{s,m}, \theta_m)$ when $\ell_n = m$ and to one otherwise. A label sits on every emission factor of its node and on every Potts factor of its edges; in the Forney form it passes through an equality node of that degree, and the gated factors are the higher-degree nodes that join the lattice to the chains (Figure 3). The graph has cycles, so Equation 2 is the Bethe approximation on it, and the exact quantities below come from enumeration on an instance small enough to enumerate—a 2×2 lattice with $M = 2$, $K = 2$, $S = 3$ has $2^4 \cdot 2^{12}$ joint states.

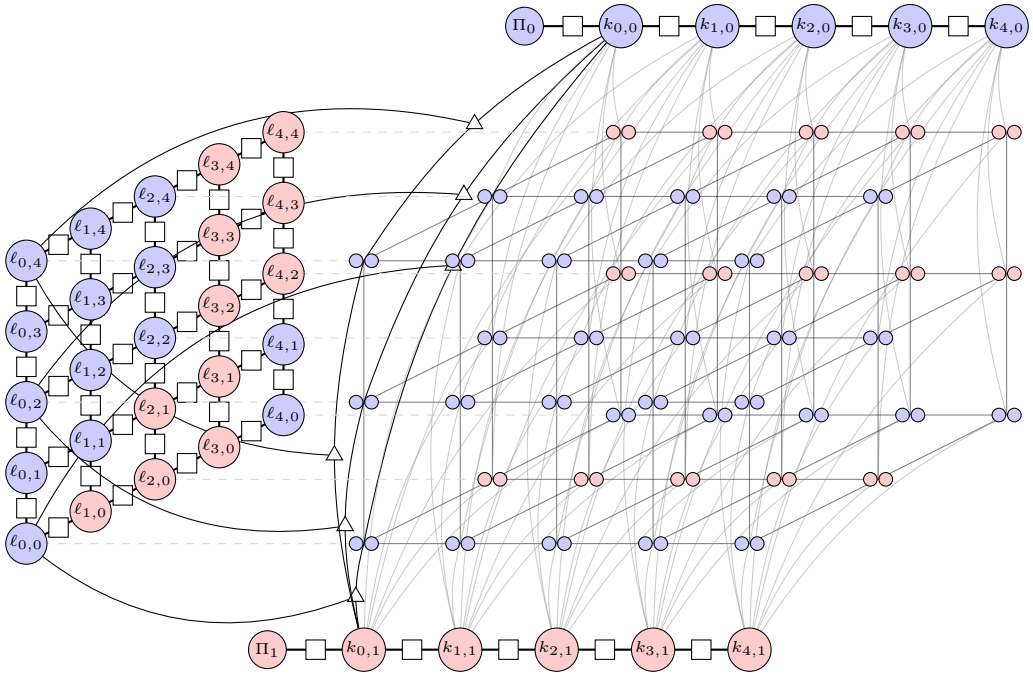

Figure 3: The coupled spatio-sequential model as a Forney-style factor graph. Each spatial node n carries a class label $\ell_n \in \{1, \ldots, M\}$ (circles on the lattice, coloured by class for $M = 2$); each class m carries a hidden chain $k_{s,m} \in \{1, \ldots, K\}$ with initial distribution Π_m (the two chains at the sides). Pairwise factors (squares) carry the Potts prior on the labels, Equation 35, and the chain prior, Equation 36. The gated emission factors (triangles) join one label to one chain state and contribute $\log p(x_{sn} \mid k_{s,m}, \theta_m)$ only when $\ell_n = m$; the observations x_{sn} are the small circles along each column. Every second observation column is drawn along each spatial axis, and an illustrative subset of the gated factors, so the structure is legible.

6.3 The estimator: block-coordinate ascent

The blocks are the emission parameters θ and the labels ℓ, with the chain states marginalized. For θ given ℓ, the gradient of the marginal likelihood is the posterior-weighted gradient of the emission terms,

$$\nabla_\theta \log p(x \mid \ell, \theta) = \sum_{n,m} \mathbb{1}[\ell_n = m] \sum_{s, k_{s,m}} \mathbb{Q}(k_{s,m} \mid \theta, \ell; x)\, \nabla_\theta \log p(x_{sn} \mid k_{s,m}, \theta_m), \tag{37}$$

where $\mathbb{Q}$ is the per-class posterior of Equation 33: this is expectation–maximization with the E step forward–backward on each class's chain over the nodes assigned to it, and t and Π_m

re-estimate in closed form. For ℓ given θ, the same posterior defines a per-node, per-class external field,

$$H_{nm}(\theta, \theta'; \ell) \equiv - \sum_{s, k_{s,m}} \mathbb{Q}(k_{s,m} \mid \theta', \ell; x) \, \log p(x_{sn} \mid k_{s,m}, \theta), \tag{38}$$

and the label update is the ground state of a Potts model in that field,

$$\ell^\star = \arg\min_{\ell} \; \beta \Big[\sum_{\langle n, n' \rangle} J_{nn'} \mathbb{1}[\ell_n \neq \ell_{n'}] + \sum_{n,m} \mathbb{1}[\ell_n = m] \, H_{nm} \Big], \tag{39}$$

which is the problem of Section 4.6, solved by alpha expansion with its bound, descended by iterated conditional modes, or sampled by an annealed cluster move with a field (Section 4.7). Each block does not decrease the joint likelihood, which is the monotonicity every expectation–maximization here is held to.

Why a cluster move. Under an effective spatial prior each class spans a contiguous region, and a single-site move must cross a region one node at a time against the prior's penalty; iterated conditional modes freezes into spurious disjoint clusters, and single-site annealing slows critically for the same reason. The Wolff move of Section 4.7 recolours a whole cluster at once, and Algorithm 2 is that move with the field of Equation 38 in its accept step. At $\beta \to 0$ it is a single-site Gibbs update; at $\beta \gg 1$ it is the greedy entrapment of iterated conditional modes; annealing in β moves between them.

Algorithm 2 One Wolff update on the labels, in the external field

draw a root ρ uniformly from the nodes; $\mu \leftarrow \ell_\rho$; $\mathcal{C} \leftarrow \{\rho\}$; queue $Q \leftarrow \{\rho\}$
while $Q \neq \varnothing$ **do**
    pop n from Q
    for each neighbour n' of n with $\ell_{n'} = \mu$ and $n' \notin \mathcal{C}$ **do**
        with probability $1 - e^{-\beta J_{nn'}}$: add n' to $\mathcal{C}$ and to Q
    end for
end while
draw a new label ν uniformly from $\{1, \ldots, M\}$
$\Delta H \leftarrow \sum_{n \in \mathcal{C}} (H_{n\nu} - H_{n\mu})$
with probability $\min\{1, e^{-\beta \Delta H}\}$: $\ell_n \leftarrow \nu$ for every $n \in \mathcal{C}$

6.4 The initializer: data-sampled burn-in

A block ascent on this joint is only as good as where it starts, and a start that ignores the graph and the emission family—k-means++ on the raw observations, say—leaves degenerate classes and false positives that the ascent then polishes. Algorithm 3 anneals in β from zero, so that early on the labels are nearly free and the chains and emission parameters are fitted to what the data alone supports, and the spatial prior is tightened only as the classes separate. Its seeding step, EmissionMixture++, is k-means++ with the emission family's own negative log-density in place of squared Euclidean distance [7]: candidates are drawn proportionally to how badly the current parameters explain them, constrained to the observations the current labels and states assign to the class being seeded. The $8(\ln K + 2)$ expected-cost bound of k-means++ is a statement about squared distance and is not inherited; what is measured instead is recovery against a uniform start at an equal budget, over seeds, reported whether it wins or not.

What pins it, and what is built. The structure and its evaluator are built: the graph of Equation 3 is constructed from the three adapters and its log-density equals the joint written

Algorithm 3 Data-sampled burn-in on the coupled graph

$\beta \leftarrow 0$; $H \leftarrow 0$; $\ell_n \sim \mathrm{Uniform}\{1, \dots, M\}$ for every node
$\theta \leftarrow$ EmissionMixture++(β, K, θ, x)
while $\beta < \beta_0$ **do**
    WolffUpdate(β, ℓ, J, H) ▷ Algorithm 2
    for each class m **do** draw the chain $k_{\cdot,m}$ by block Gibbs given ℓ, θ
    end for
    for each state k **do** $\theta_k \leftarrow$ EmissionMixture++$(\beta, K, \theta, x_k, \ell)$ ▷ state-constrained
    end for
    $\mathbb{Q} \leftarrow$ forward–backward per class (Equation 33); M step for θ, t, Π
    $H_{nm} \leftarrow$ Equation 38
    $\beta \leftarrow \beta + \Delta\beta$
end while
return ℓ, θ

out on every assignment of an enumerable instance. The fixture with its enumeration oracle, the E step and the field, the block ascent with the three label solvers, and the initializer are the remaining parts, each with its own pin: label and state marginals of the simulator against enumeration; the gradient identity Equation 37 against central differences; the joint non-decreasing across every block; label accuracy up to permutation against the planted labels at an equal budget of sweeps, for each of the three label solvers, so the claim that a cluster move escapes what iterated conditional modes freezes into is a measurement here and not a premise.

7 Problem Statement: LDPC Decoding

7.1 The model

A binary linear code of length n is the null space of a parity-check matrix $\mathbf{H} \in \{0,1\}^{m \times n}$ over GF(2): the codewords are the c with $\mathbf{H}c = 0$, there are 2^k of them with $k = n - \operatorname{rank} \mathbf{H}$, and the rate is k/n. A low-density code makes $\mathbf{H}$ sparse. Gallager's regular (d_v, d_c) ensemble [30] puts d_v ones in every column and d_c in every row: $\mathbf{H}$ is d_v bands of n/d_c rows, the first band's row i covering bits id_c to $(i+1)d_c - 1$ and every other band the first under a uniformly random permutation of the columns, so n is a multiple of d_c and $m = nd_v/d_c$. The rows of one band sum to the all-ones vector, so at least $d_v - 1$ rows are dependent and $k \geq n - m + d_v - 1$. The instance this repository carries is $(3, 6)$ at $n = 19{,}998$, the multiple of six nearest the 20,000 the ticket names: $m = 9{,}999$ checks, 59,994 nonzeros, design rate 1/2.

A codeword is sent through a memoryless binary-input channel and y is received. Three channels are supported, each seen only through its log-likelihood ratio,

$$L_i = \log \frac{p(y_i \mid c_i = 0)}{p(y_i \mid c_i = 1)} = \begin{cases} (1 - 2y_i) \log \dfrac{1-p}{p} & \text{binary symmetric, flip probability } p \\ 0 \text{ if erased, else } \pm\infty \text{ with the sign of } 1 - 2y_i & \text{binary erasure, erasure probability} \\ 2y_i/\sigma^2 & \text{binary-input Gaussian, } y_i = (1-2c \end{cases} \tag{40}$$

positive when the received symbol favours zero. The posterior over the code is then

$$p(c \mid y) \;\propto\; \mathbb{1}[\mathbf{H}c = 0] \, \exp(-c \cdot L), \tag{41}$$

since $\log p(y \mid c) = \sum_i \log p(y_i \mid 0) - \sum_i c_i L_i$ and the first sum is the same for every word. Decoding asks two different questions of Equation 41: the *bitwise* MAP, each bit at the mode of its own marginal, which minimizes the bit error rate and need not be a codeword; and the

blockwise MAP, the single most probable codeword, which minimizes the block error rate. They are different answers, as Section 5 says of the chain, and the fixture that separates them is a test.

7.2 As a factor graph

The variables are the bits, $x_i \equiv c_i \in \{0,1\}$. The factors are a unary $\psi_i(c_i) = e^{-c_i L_i}$ per bit, the channel folded in as an observation is everywhere in this document, and a parity factor $\psi_a(c_{\partial a}) = \mathbb{1}[\bigoplus_{i\in\partial a} c_i = 0]$ per row of $\mathbf{H}$, a hard constraint of degree d_c whose log table is zero on even-parity assignments and $-\infty$ elsewhere. This is the Tanner graph of the code; a bit sits on d_v parity factors and one unary, so in the Forney form every bit passes through an equality node of degree $d_v + 1$. The graph has cycles—a random draw of the ensemble does not avoid short ones—so Equation 2 is the Bethe approximation on it, and the exact quantities below come from enumerating the 2^k codewords where $2^k \le 200{,}000$ (a $(3,6)$ code at $n = 24$ has $k \ge 14$) and from the tree schedule on a cycle-free code, where message passing is exact.

7.3 The algorithm: sum-product in the log domain

A binary variable's message is one number, its log-odds, and Equation 2 on a parity factor has a closed form [30, 53, 72]. From bit i to check a, and from check a to bit i,

$$m_{i\to a} = L_i + \sum_{b\in\partial i\setminus a} m_{b\to i}, \qquad m_{a\to i} = 2\operatorname{artanh} \prod_{j\in\partial a\setminus i} \tanh\frac{m_{j\to a}}{2}, \qquad \tilde{L}_i = L_i + \sum_{b\in\partial i} m_{b\to i}, \tag{42}$$

where the check update follows from $\tanh(L/2) = p(0) - p(1)$ and the fact that the parity of independent bits has a difference of probabilities equal to the product of theirs. The hard decision $\hat{c}_i = \mathbb{1}[\tilde{L}_i < 0]$ is the bitwise MAP under the Bethe beliefs. Replacing the sum by a maximum in Equation 2 gives max-product, whose log-domain check update is

$$m_{a\to i} = \Big(\prod_{j\in\partial a\setminus i} \operatorname{sgn} m_{j\to a}\Big) \min_{j\in\partial a\setminus i} |m_{j\to a}|, \tag{43}$$

the min-sum decoder, whose decision is the blockwise MAP and is exact on a cycle-free code with a unique maximum; on a loopy code it is the cheaper and the weaker of the two, and the gap is measured rather than assumed.

Three details are load-bearing in the implementation. The schedule is flooding: every check reads the bit messages of the previous half-iteration and every bit reads the check messages just written, one Jacobi sweep per direction over all edges at once, which is the schedule the general implementation runs and so the one it is compared against. The loop stops at the first iteration whose hard decision has every bit decided and satisfies $\mathbf{H}\hat{c} = 0$, the syndrome check, and otherwise at an iteration cap, which is a decoding failure and the event a block error rate counts. And messages are clipped at $\pm L_{\max}$ with $L_{\max} = 30$ before the check update, because a certain bit's ratio is infinite on the erasure channel and the leave-one-out sums in Equation 42 are formed by subtraction: $\tanh(L_{\max}/2)$ is below one by 1.9×10^{-13} in double precision, and a belief formed under the cap differs from the exact one by less than $e^{-L_{\max}}$ in probability, which the oracle tests absorb.

7.4 The all-zero codeword

Every simulation past the reach of an encoder sends the zero word, and the argument that this loses nothing is standard [72, Sec. 4.1]. Each channel of Equation 40 is output symmetric: sending $c_i = 1$ instead of 0 negates L_i in distribution, and on the symmetric and erasure channels it negates it on every realization, since a flip or an erasure is applied to the bit whatever its value. Both check updates are odd in each argument and the bit update is linear, so negating the

ratios at the bits where $c_i = 1$ negates every message and every posterior at exactly those bits, and the hard decision under c is the hard decision under 0 with c added: the error pattern $c \oplus \hat{c}$ has the same distribution whatever codeword was sent. Bit and block error rates measured on the zero word are therefore the rates on any codeword. Where an encoder exists—Gauss–Jordan elimination over GF(2) to a systematic form, at $n \leq 512$—the identity is checked per realization on a non-trivial codeword, so the shortcut is itself pinned.

What pins it. Four oracles, none sharing code with the decoder. On a cycle-free code the tree schedule of the general implementation and the enumeration over all 2^k codewords are exact, and the decoder equals both; min-sum equals the general max-product there and returns the enumerated maximum-likelihood codeword, with the margin over the runner-up pinned so that no tie is being broken. On loopy codes the decoder and the general damped flooding are the same Bethe approximation and reach the same fixed point, which is agreement and not truth. The encoder is pinned by $\mathbf{H}c = 0$ and the offsets layout by the dense product it stands in for. And at size, where no enumeration reaches, the closed form is density evolution (Appendix A.7): on the erasure channel the $(3, 6)$ ensemble has a threshold $\epsilon^\star = 0.4294$ below which the infinite-length decoder resolves every erasure and above which it does not, and the 19,998-bit code is held to bracketing it.

8 The Properties That Referee Each Method

An algorithm is accepted here against a property that is true independently of this implementation. The properties are not interchangeable, and which one applies is a statement about the method rather than a matter of convenience. Each *What pins it* paragraph above names one of the five kinds below, and the regression suite marks every check with the kind it is: an oracle, a simulated truth, a mathematical invariant, an edge case, or a structural property of the code. A check that fits none of them is either a missing category or a test with nothing to assert.

8.1 Exact answers, where an oracle exists

Where a quantity can be computed a second way that shares no recursion with the first, equality is asserted to a stated tolerance. Direct marginalization over k^{n-2} internal assignments referees pruning; a sum over k^T paths referees the forward recursion; exhaustive enumeration over $q^{|V|}$ configurations referees a Potts marginal; enumeration of $(2n-5)!!$ topologies referees a search. Every one is exponential, and that is what makes it an oracle rather than a second opinion. **Two implementations agreeing prove nothing if both are wrong**, so an accelerated method is pinned against the reference and never against another acceleration.

8.2 Structural invariants, where no oracle reaches

Past the sizes enumeration allows, what remains are properties the model must satisfy whatever the answer is:

- rows of a transition matrix sum to one, and $\mathbf{P}(0) = \mathbf{I}$;
- a reversible model satisfies detailed balance, $\pi_i \mathbf{Q}_{ij} = \pi_j \mathbf{Q}_{ji}$;
- the *pulley principle*: on a reversible model the likelihood is invariant to where the root is placed along a branch, so a rooted computation and its re-rooting must agree exactly;
- a gradient matches central differences, relative to the gradient's own norm—entrywise fails wherever an entry is zero, and an absolute bound does not transfer across data sizes;
- an expectation-maximization iterate increases the likelihood monotonically.

None of these establishes that a method is right. Each establishes that a specific way of being wrong did not happen, which is a weaker and more defensible claim.

8.3 Recovery, which is the acceptance test for a fit

A likelihood that increases proves the optimizer runs, not that the model is right. The acceptance test is to fit data simulated from known parameters and require the intervals to cover the truth at their nominal rate over independent replicates. Two qualifications are load-bearing. Where a model has an exact symmetry—permuting the hidden states of an HMM or the components of a mixture leaves the likelihood unchanged—recovery is stated *up to that symmetry*, and the alignment is part of the test. And coverage is measured as a function of how identifiable the instance is, because a fixture chosen where everything is well separated is a test of the fixture.

8.4 Where the likelihood has no maximum, or no curvature

Two failures are properties of models rather than defects, and both must be handled before they are discovered.

An *unbounded* likelihood has no maximum to find: put a Gaussian component's mean on a single observation and let its variance go to zero and the density there diverges. A fit that converges there was stopped by its starting point or by a floor, and which one must be knowable—so the bound is explicit, derived from the data rather than chosen, and reaching it is a refusal rather than a clamp.

A *flat* likelihood has a maximum that runs to the boundary: an overdispersed count model approaches its Poisson limit as the dispersion grows, and past the point where the overdispersion is smaller than the sampling noise on measuring it, the data cannot tell one value from another. The two failures are mirror images, and they announce themselves differently—the first as a divergence, the second as a singular information matrix—which is why each is measured rather than inferred from the other.

8.5 Agreement across implementations is a tolerance

An accelerated implementation is pinned against the reference within a declared *relative* tolerance, never bitwise. Relative, because a log-likelihood is a sum over sites and an absolute bound fixed at one size does not transfer to another. Keyed on the *lowest* precision in the comparison, because a single-precision device cannot be held to a double-precision bound, and one bound loose enough for single precision would let a broken double-precision backend pass. A discrepancy inside the tolerance is not a defect and is not corrected; one outside it is not a tolerance to loosen. The values themselves live beside the measurements they are derived from, not here.

8.6 Published constants, and asymptotic results

Some claims come from outside entirely: the $(2n-5)!!$ count, Goemans–Williamson's 0.87856 for maximum cut, alpha expansion's factor of two, the $8(\ln k + 2)$ bound on k-means++ seeding. These referee an implementation at any size. **No asymptotic result is testable at the sizes an exact oracle allows**, however: a threshold or a limit holds as the problem grows and says nothing where enumeration can check it. Such a result is reported for orientation and the per-instance property is what is asserted.

9 Relevance to the Roadmap

The three problem classes are not three applications. They are chosen so that each abstraction is tested against something that is not the model it was extracted from: a likelihood engine

justified only by trees would be a phylogenetics library, and an optimizer justified only by an HMM would encode a chain.

The order the milestones take follows the dependency between the properties above. Simulation and its ground truth come first, because every later claim is checked against data whose generating parameters are known. The exact evaluators come next, because they are the oracles the accelerated ones are pinned to. Continuous optimization follows, since recovery is the acceptance test and needs both. Discrete search and learned proposals come last, because their acceptance criterion—reaching the optimum enumeration identifies—only exists once enumeration does.

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[2] Gautam Altekar, Sandhya Dwarkadas, John P. Huelsenbeck, and Fredrik Ronquist. Parallel Metropolis coupled Markov chain Monte Carlo for Bayesian phylogenetic inference. *Bioinformatics*, 20(3):407–415, 2004.

[3] Shun-ichi Amari. *Information Geometry and Its Applications*. Springer, Tokyo, 2016.

[4] Maria Chiara Angelini and Federico Ricci-Tersenghi. Modern graph neural networks do worse than classical greedy algorithms in solving combinatorial optimization problems like maximum independent set. *Nature Machine Intelligence*, 5:29–31, 2023.

[5] Tiago Antão. *Fast Python: High Performance Techniques for Large Datasets*. Manning, 2023.

[6] Thomas Anthony, Zheng Tian, and David Barber. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[7] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035, 2007.

[8] Dana Azouri, Shiran Abadi, Yishay Mansour, Itay Mayrose, and Tal Pupko. Harnessing machine learning to guide phylogenetic-tree search algorithms. *Nature Communications*, 12: 1983, 2021.

[9] Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017.

[10] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d'horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.

[11] Michael Betancourt. A conceptual introduction to Hamiltonian Monte Carlo. *arXiv preprint arXiv:1701.02434*, 2017.

[12] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006.

[13] Richard E. Blahut. *Algebraic Codes for Data Transmission*. Cambridge University Press, Cambridge, 2003.

[14] Jim Blandy, Jason Orendorff, and Leonora F. S. Tindall. *Programming Rust: Fast, Safe Systems Development*. O'Reilly Media, 2nd edition, 2021.

[15] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[16] Yuri Boykov and Vladimir Kolmogorov. An experimental comparison of min-cut/max-flow algorithms for energy minimization in vision. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(9):1124–1137, 2004.

[17] Yuri Boykov, Olga Veksler, and Ramin Zabih. Fast approximate energy minimization via graph cuts. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 23(11): 1222–1239, 2001.

[18] Randal E. Bryant and David R. O'Hallaron. *Computer Systems: A Programmer's Perspective*. Pearson, 3rd edition, 2015.

[19] Phillip Compeau and Pavel Pevzner. *Bioinformatics Algorithms: An Active Learning Approach*. Active Learning Publishers, 2015.

[20] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, Massachusetts, 4th edition, 2022.

[21] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley, 2 edition, 2006.

[22] Richard Durbin, Sean R. Eddy, Anders Krogh, and Graeme Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press, Cambridge, 1998.

[23] David J. Earl and Michael W. Deem. Parallel tempering: Theory, applications, and new perspectives. *Physical Chemistry Chemical Physics*, 7:3910–3916, 2005.

[24] Robert G. Edwards and Alan D. Sokal. Generalization of the Fortuin–Kasteleyn–Swendsen–Wang representation and Monte Carlo algorithm. *Physical Review D*, 38(6):2009–2012, 1988.

[25] Joseph Felsenstein. Evolutionary trees from DNA sequences: A maximum likelihood approach. *Journal of Molecular Evolution*, 17(6):368–376, 1981.

[26] Joseph Felsenstein. *Inferring Phylogenies*. Sinauer Associates, Sunderland, Massachusetts, 2004.

[27] Walter M. Fitch. Toward defining the course of evolution: Minimum change for a specific tree topology. *Systematic Zoology*, 20(4):406–416, 1971.

[28] G. David Forney. Codes on graphs: Normal realizations. *IEEE Transactions on Information Theory*, 47(2):520–548, 2001.

[29] Brendan J. Frey. *Graphical Models for Machine Learning and Digital Communication*. MIT Press, Cambridge, Massachusetts, 1998.

[30] Robert G. Gallager. Low-density parity-check codes. *IRE Transactions on Information Theory*, 8(1):21–28, 1962.

[31] Charles J. Geyer. Markov chain Monte Carlo maximum likelihood. In *Computing Science and Statistics: Proceedings of the 23rd Symposium on the Interface*, pages 156–163, 1991.

[32] Charles J. Geyer. Practical Markov chain Monte Carlo. *Statistical Science*, 7(4):473–483, 1992.

[33] Michel X. Goemans and David P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6):1115–1145, 1995.

[34] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, Massachusetts, 2016.

[35] Micha Gorelick and Ian Ozsvald. *High Performance Python*. O'Reilly Media, 2 edition, 2020.

[36] D. M. Greig, B. T. Porteous, and A. H. Seheult. Exact maximum a posteriori estimation for binary images. *Journal of the Royal Statistical Society: Series B*, 51(2):271–279, 1989.

[37] Nikolaus Hansen and Andreas Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195, 2001.

[38] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[39] Matthew D. Hoffman and Andrew Gelman. The No-U-Turn sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15: 1593–1623, 2014.

[40] Daniel Hsu, Sham M. Kakade, and Tong Zhang. A spectral algorithm for learning hidden Markov models. *Journal of Computer and System Sciences*, 78(5):1460–1480, 2012.

[41] Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 4th edition, 2022.

[42] Elias Khalil, Hanjun Dai, Yuyu Zhang, Bistra Dilkina, and Le Song. Learning combinatorial optimization algorithms over graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[43] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[44] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, Cambridge, Massachusetts, 2009.

[45] Vladimir Kolmogorov and Ramin Zabih. What energy functions can be minimized via graph cuts? *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(2):147–159, 2004.

[46] Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations*, 2019.

[47] Alexey M. Kozlov, Diego Darriba, Tomáš Flouri, Benoit Morel, and Alexandros Stamatakis. RAxML-NG: A fast, scalable and user-friendly tool for maximum likelihood phylogenetic inference. *Bioinformatics*, 35(21):4453–4455, 2019.

[48] Werner Krauth. *Statistical Mechanics: Algorithms and Computations*. Oxford University Press, 2006.

[49] Frank R. Kschischang, Brendan J. Frey, and Hans-Andrea Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47(2):498–519, 2001.

[50] David P. Landau and Kurt Binder. *A Guide to Monte Carlo Simulations in Statistical Physics*. Cambridge University Press, 4 edition, 2014.

[51] Maxim Lapan. *Deep Reinforcement Learning Hands-On*. Packt Publishing, 2nd edition, 2020.

[52] Jonathan Machta. Population annealing with weighted averages: A Monte Carlo method for rough free-energy landscapes. *Physical Review E*, 82:026704, 2010.

[53] David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, Cambridge, 2003.

[54] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.

[55] Nina Mazyavkina, Sergey Sviridov, Sergei Ivanov, and Evgeny Burnaev. Reinforcement learning for combinatorial optimization: A survey. *Computers & Operations Research*, 134: 105400, 2021.

[56] Marc Mézard and Andrea Montanari. *Information, Physics, and Computation*. Oxford University Press, Oxford, 2009.

[57] Bui Quang Minh, Heiko A. Schmidt, Olga Chernomor, Dominik Schrempf, Michael D. Woodhams, Arndt von Haeseler, and Robert Lanfear. IQ-TREE 2: New models and efficient methods for phylogenetic inference in the genomic era. *Molecular Biology and Evolution*, 37 (5):1530–1534, 2020.

[58] Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023.

[59] Eliya Nachmani, Elad Marciano, Loren Lugosch, Warren J. Gross, David Burshtein, and Yair Be'ery. Deep learning methods for improved decoding of linear codes. *IEEE Journal of Selected Topics in Signal Processing*, 12(1):119–131, 2018.

[60] Radford M. Neal. MCMC using Hamiltonian dynamics. In Steve Brooks, Andrew Gelman, Galin L. Jones, and Xiao-Li Meng, editors, *Handbook of Markov Chain Monte Carlo*, pages 113–162. Chapman and Hall/CRC, 2011.

[61] M. E. J. Newman and G. T. Barkema. *Monte Carlo Methods in Statistical Physics*. Oxford University Press, Oxford, 1999.

[62] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, Cambridge, 10th anniversary edition, 2010.

[63] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.

[64] Antonio Ortega. *Introduction to Graph Signal Processing*. Cambridge University Press, Cambridge, 2022.

[65] Lior Pachter and Bernd Sturmfels, editors. *Algebraic Statistics for Computational Biology*. Cambridge University Press, Cambridge, 2005.

[66] Christos H. Papadimitriou and Kenneth Steiglitz. *Combinatorial Optimization: Algorithms and Complexity*. Dover, 1998.

[67] Max B. Paulus, Dami Choi, Daniel Tarlow, Andreas Krause, and Chris J. Maddison. Gradient estimation with stochastic softmax tricks. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[68] Judea Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, San Mateo, California, 1988.

[69] Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, Cambridge, Massachusetts, 2023.

[70] Luciano Ramalho. *Fluent Python*. O'Reilly Media, 2 edition, 2022.

[71] Sebastian Raschka. *Build a Large Language Model (From Scratch)*. Manning Publications, 2024.

[72] Tom Richardson and Rüdiger Urbanke. *Modern Coding Theory*. Cambridge University Press, Cambridge, 2008.

[73] Kenneth H. Rosen. *Discrete Mathematics and Its Applications*. McGraw-Hill, 8th edition, 2019.

[74] David Sankoff. Minimal mutation trees of sequences. *SIAM Journal on Applied Mathematics*, 28(1):35–42, 1975.

[75] Ulrich Schollwöck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics*, 326(1):96–192, 2011.

[76] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.

[77] Martin J. A. Schuetz, J. Kyle Brubaker, and Helmut G. Katzgraber. Combinatorial optimization with physics-inspired graph neural networks. *Nature Machine Intelligence*, 4: 367–377, 2022.

[78] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.

[79] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[80] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 550:354–359, 2017.

[81] David Speyer and Bernd Sturmfels. The tropical Grassmannian. *Advances in Geometry*, 4 (3):389–411, 2004.

[82] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, Massachusetts, 2nd edition, 2018.

[83] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112 (1–2):181–211, 1999.

[84] Robert H. Swendsen and Jian-Sheng Wang. Replica Monte Carlo simulation of spin-glasses. *Physical Review Letters*, 57(21):2607–2609, 1986.

[85] Robert H. Swendsen and Jian-Sheng Wang. Nonuniversal critical dynamics in monte carlo simulations. *Physical Review Letters*, 58(2):86–88, 1987.

[86] Martin J. Wainwright and Michael I. Jordan. Graphical models, exponential families, and variational inference. *Foundations and Trends in Machine Learning*, 1(1–2):1–305, 2008.

[87] Martin J. Wainwright, Tommi S. Jaakkola, and Alan S. Willsky. A new class of upper bounds on the log partition function. *IEEE Transactions on Information Theory*, 51(7): 2313–2335, 2005.

[88] Martin J. Wainwright, Tommi S. Jaakkola, and Alan S. Willsky. MAP estimation via agreement on trees: Message-passing and linear programming. *IEEE Transactions on Information Theory*, 51(11):3697–3717, 2005.

[89] Chris Whidden and Frederick A. Matsen. Quantifying MCMC exploration of phylogenetic tree space. *Systematic Biology*, 64(3):472–491, 2015.

[90] Ulli Wolff. Collective monte carlo updating for spin systems. *Physical Review Letters*, 62 (4):361–364, 1989.

[91] Ziheng Yang. *Molecular Evolution: A Statistical Approach*. Oxford University Press, 2014.

[92] Jonathan S. Yedidia, William T. Freeman, and Yair Weiss. Constructing free-energy approximations and generalized belief propagation algorithms. *IEEE Transactions on Information Theory*, 51(7):2282–2312, 2005.

[93] Cheng Zhang and Frederick A. Matsen. Generalizing tree probability estimation via Bayesian networks. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[94] Cheng Zhang and Frederick A. Matsen. Variational Bayesian phylogenetic inference. In *International Conference on Learning Representations*, 2019.

A Derivations Cited From the Text

Each of these is standard; each is here because a claim in the main text depends on it, and the main text cites rather than derives, for the reader the preface names.

A.1 Pruning is the marginalization over internal states

Root $\mathcal{T}$ at ρ and write z_u for the state at node u, so the leaves' states at one site are the data $\mathbf{D}_s$ and the internal states are summed. Along each branch the state evolves by the Markov process of Section 3, and the branches are conditionally independent given the state at their common node, so the joint over every node is $p(z) = \pi_{z_\rho} \prod_{v \neq \rho} \mathbf{P}_{z_{\mathrm{pa}(v)} z_v}(t_v)$, and the site likelihood is its sum over the internal states with the leaves clamped—the summand of Equation 17, with k^{n-2} terms. Define $L_u(i) = p(\mathbf{D}_s^{(u)} \mid z_u = i)$, the probability of the data at the leaves below u given u's state. The subtrees hanging from u's children share no variable and, given z_u, no factor, so the sum over their states factorizes into one sum per child:

$$L_u(i) = \prod_{v \in \mathrm{children}(u)} p(\mathbf{D}_s^{(v)} \mid z_u = i) = \prod_{v \in \mathrm{children}(u)} \sum_{j \in \mathcal{A}} \mathbf{P}_{ij}(t_v)\, p(\mathbf{D}_s^{(v)} \mid z_v = j),$$

the second equality conditioning on z_v and using that the data below v depends on z_u only through z_v. The inner probability is $L_v(j)$, which is Equation 18; at a leaf $L_u(i) = \mathbb{1}[z_u = i]$ because the state is observed; and summing the root state against its prior gives Equation 19. Nothing in the argument fixes the order of the children or the depth of the tree, so the recursion is a post-order traversal in which each node's sum is done once per parent state: $O(nk^2)$ per site against k^{n-2} terms. Each L_u is linear in each L_v, so rescaling L_u by a positive c_u scales the site likelihood by $\prod_u c_u$ and nothing else, which is why the logarithm of the scaling may be accumulated apart from the recursion and the rescaled recursion is a transformation of this one rather than a second algorithm [25, 26].

A.2 Forward–backward is the same marginalization on a chain

With the factors of Section 5 the joint over the hidden path and the observations is $p(\mathbf{Z}, \mathbf{D}) = \pi_{z_1}\, \mathcal{E}_{z_1}(y_1) \prod_{t>1} \mathbf{A}_{z_{t-1} z_t}\, \mathcal{E}_{z_t}(y_t)$, and the evidence sums it over k^T paths. Define $\alpha_t(i) = p(y_1, \dots, y_t, z_t = i)$. Conditioning on z_{t-1}, and using that $y_1, \dots, y_{t-1}$ depend on z_t only through z_{t-1}, splits the sum over the first $t-1$ states from the rest: $\alpha_t(i) = \big[\sum_j \alpha_{t-1}(j)\, \mathbf{A}_{ji}\big]\, \mathcal{E}_i(y_t)$, with $\alpha_1(i) = \pi_i\, \mathcal{E}_i(y_1)$, and the evidence is $\sum_i \alpha_T(i)$: Equation 32. Define $\beta_t(i) = p(y_{t+1}, \dots, y_T \mid z_t = i)$, so $\beta_T = 1$; conditioning on z_{t+1} in the same way gives the backward recursion of Equation 33. The two meet at any position: $p(z_t = i, \mathbf{D}) = \alpha_t(i)\, \beta_t(i)$, because the observations before t and after t are independent given z_t, and dividing by the evidence gives the posterior of Equation 33; the same split one step wider gives the pairwise posterior $p(z_{t-1} = i, z_t = j \mid \mathbf{D}) = \alpha_{t-1}(i)\, \mathbf{A}_{ij}\, \mathcal{E}_j(y_t)\, \beta_t(j)/p(\mathbf{D})$, the sufficient statistic of the expectation step.

The chain is the caterpillar tree of Section 5 rooted at z_1: each z_t has one child z_{t+1} carrying the branch factor $\mathbf{A}$ and one observed leaf y_t carrying the emission. Appendix A.1 then gives $L_{z_t}(i) = \mathcal{E}_i(y_t) \sum_j \mathbf{A}_{ij}\, L_{z_{t+1}}(j)$, so $L_{z_t}(i) = \mathcal{E}_i(y_t)\, \beta_t(i)$ and Equation 19 is $\sum_i \pi_i\, \mathcal{E}_i(y_1)\, \beta_1(i) = \sum_i \alpha_1(i)\, \beta_1(i) = p(\mathbf{D})$: pruning on the chain is the backward pass, and the forward pass is the same recursion run toward the other end with the prior entering as a leaf factor at z_1. Both passes in the logarithm are the rescaled recursion of Appendix A.1 with the scale taken at every step. Replacing each sum by a maximum leaves every step of the argument intact, since a maximum distributes over a product of non-negative factors as a sum does, and gives Equation 34 [22, 44].

A.3 Sum-product on a tree, and its loopy stationary point

Both derivations above are one fact about a tree-structured factor graph. Take an edge between factor a and variable i and remove it; because the graph is a tree, the component containing a is a subtree $\mathcal{T}_{a \to i}$ that meets the rest only at x_i. Define the message from a to i as the partition function of that subtree with x_i clamped, $m_{a \to i}(x_i) = \sum_{x_{\mathcal{T}_{a \to i}} \setminus x_i} \prod_{b \in \mathcal{T}_{a \to i}} \psi_b(x_{\partial b})$, and $m_{i \to a}$ likewise for the subtree on i's side. The subtrees hanging from a's other variables $j \in \partial a \setminus i$ are disjoint, so the sum over their variables factorizes into a product of their partition functions, each a function of its own x_j; and the subtrees hanging from i's other factors share only x_i, so their product is $m_{i \to a}(x_i)$. That is Equation 2, with the messages at a leaf variable or leaf factor being 1 and the factor itself. At any variable the whole graph is the union of the subtrees on its factors, so $Z = \sum_{x_i} \prod_{a \in \partial i} m_{a \to i}(x_i)$ and the marginal is that product normalized; at a factor, $p(x_{\partial a}) \propto \psi_a(x_{\partial a}) \prod_{i \in \partial a} m_{i \to a}(x_i)$. A message depends only on messages from further from any chosen root, so one leaf-to-root pass computes every message toward it and one root-to-leaf pass every message away, and every marginal follows: two passes, each factor summed once per direction [68, 49, 44]. Pruning is this with a node's state as the variable, a branch matrix as the factor between parent and child, the prior as a factor on the root and a leaf's observation as an indicator factor: $L_u(i)$ is the product of the factor-to-variable messages from below, which is the variable-to-factor message $m_{u \to \psi_u}$ that u sends up its own branch. Forward–backward is this on

the chain. The argument used only that sum distributes over product, so max-product follows by replacing every sum with a maximum, exact on a tree when the maximum is unique.

On a graph with a cycle no edge separates the graph into two subtrees, so the message has no such definition; iterating Equation 2 regardless defines a map on messages, and a fixed point of that map is a stationary point of the Bethe free energy, which Appendix A.4 derives for the pairwise case of Equation 24 and which reads, for factors of any degree,

$$F_{\text{Bethe}} = \sum_a \sum_{x_{\partial a}} b_a \left(\log b_a - \log \psi_a \right) - \sum_i (d_i - 1) \sum_{x_i} b_i \log b_i, \tag{44}$$

with d_i the number of factors on i. On a tree $p(x) = \prod_a b_a / \prod_i b_i^{d_i - 1}$ exactly, the beliefs being the true marginals, so Equation 44 is $-\log Z$ and the stationary point is the answer of the two passes; on a loopy graph the factorization fails and the fixed point is the approximation [92, 56].

A.4 The Bethe free energy is stationary at a sum-product fixed point

Write the beliefs of a pairwise graph as b_i and b_{ij} and constrain them to be normalized and locally consistent, $\sum_{\sigma_j} b_{ij} = b_i$. Introducing a multiplier $\lambda_{ij}(\sigma_i)$ for each consistency constraint and setting the derivative of Equation 24 to zero gives $b_{ij} \propto \psi_{ij}\, e^{\lambda_{ij}(\sigma_i) + \lambda_{ji}(\sigma_j)}$ and $b_i \propto \psi_i\, e^{\sum_{j \in \partial i} \lambda_{ji}(\sigma_i)/(d_i - 1)}$; identifying $e^{\lambda_{ji}(\sigma_i)}$ with $\prod_{l \in \partial i \setminus j} m_{l \to i}(\sigma_i)$ turns the consistency constraint into Equation 23. So a fixed point of belief propagation is a stationary point of the Bethe free energy and conversely; on a tree the Bethe free energy is the exact variational free energy, which is why the fixed point there gives $-\log Z$ [92]. The argument uses only that each b_a is a function of the variables its factor touches, so it holds for factors of any degree, with Equation 44 in place of Equation 24 and $d_i - 1$ counting the factors on i beyond the first (Appendix A.3).

A.5 Detailed balance for a cluster move in a field

In the Edwards–Sokal representation the joint over spins and bonds is $p(\sigma, \omega) \propto \prod_{\langle i,j \rangle : \omega_{ij} = 1} (1 - e^{-\beta J_{ij}})\, \delta(\sigma_i, \sigma_j) \prod_{\omega_{ij} = 0} e^{-\beta J_{ij}} \prod_i e^{\beta h_i(\sigma_i)}$, whose spin marginal is the Boltzmann distribution of Equation 22. Drawing bonds given spins and then recolouring a bond cluster $\mathcal{C}$ uniformly are both Gibbs steps at zero field, so the move preserves p there. With a field the recolouring is no longer a Gibbs step: the ratio of the target before and after recolouring $\mathcal{C}$ from μ to ν is $\exp(\beta \sum_{i \in \mathcal{C}} [h_i(\nu) - h_i(\mu)])$, the bond configuration being unchanged, and a uniform proposal accepted with the Metropolis probability of Equation 29 satisfies detailed balance with respect to that ratio. Omitting the accept step gives a chain that runs and mixes and converges to the zero-field distribution, which is why the distributional test is run with a field as well as without.

A.6 The exchange ratio in parallel tempering

The replicas' joint target is the product $\prod_r \exp(-\beta_r E(\sigma_r))$. Swapping the configurations of replicas i and j changes it by $\exp(-\beta_i E_j - \beta_j E_i) / \exp(-\beta_i E_i - \beta_j E_j) = \exp[(\beta_i - \beta_j)(E_i - E_j)]$, and the proposal is symmetric, so the Metropolis acceptance is Equation 30. Each replica's marginal under the joint is its own tempered Boltzmann distribution whatever the exchanges do, which is the property the per-replica goodness-of-fit test asserts.

A.7 Density evolution on the erasure channel

On a cycle-free (d_v, d_c) graph the messages arriving at a node are independent, so the erasure probability of a message is a number rather than a distribution and Equation 42 reduces to a scalar recursion. A check-to-bit message is known only if every other incoming bit message

is known; a bit-to-check message is erased only if the channel and every other incoming check message are. With x_ℓ the probability that a bit-to-check message is erased after ℓ iterations,

$$x_0 = \epsilon, \qquad x_\ell = \epsilon\big(1 - (1 - x_{\ell-1})^{d_c-1}\big)^{d_v-1}, \tag{45}$$

the recursion Richardson and Urbanke [72, Sec. 3.12] derives and names density evolution. The right-hand side is increasing in $x_{\ell-1}$ and in ϵ, so x_ℓ is non-increasing and its limit is the largest fixed point in $[0, \epsilon]$; the threshold $\epsilon^\star$ is the largest ϵ for which that fixed point is zero, found by bisection, and for $(3, 6)$ it is 0.4294, with 0.3834 for $(4, 8)$ and 0.5176 for $(3, 5)$. A finite code is not cycle-free, but the neighbourhood of a bit to depth ℓ is a tree with probability tending to one as n grows at fixed ℓ, which is why the recursion is the limit of the flooding decoder and why the code at size is held to bracketing $\epsilon^\star$ rather than to a per-draw agreement—the rule `sim/CLAUDE.md` states for every asymptotic result.

A.8 The delta method for an interval at a fit

At a maximum $\hat{\boldsymbol{\theta}}$ of the log-likelihood, the observed information $\mathcal{I} = -\nabla^2 \log p(\mathbf{D} \mid \hat{\boldsymbol{\theta}})$ gives the asymptotic covariance $\Sigma = \mathcal{I}^{-1}$ of $\hat{\boldsymbol{\theta}}$. A constrained parameter $\phi(\boldsymbol{\theta})$ is a smooth function of $\boldsymbol{\theta}$, so to first order $\mathrm{Var}(\phi(\hat{\boldsymbol{\theta}})) \approx J \Sigma J^\top$ with $J = \partial\phi/\partial\boldsymbol{\theta}$ at $\hat{\boldsymbol{\theta}}$. The statement is asymptotic in the sample size and requires $\mathcal{I}$ to be invertible; a singular or ill-conditioned information is a parameter the data does not identify, and the interval is refused rather than reported (Section 8).

B Derivations for the Actor–Critic

Any state-dependent baseline leaves the estimator unbiased. For a function b of the state alone, $\mathbb{E}_{a\sim\pi(\cdot|s)}[\nabla \log \pi(a \mid s)\, b(s)] = b(s) \sum_a \nabla\pi(a \mid s) = b(s)\, \nabla \sum_a \pi(a \mid s) = 0$, so subtracting $b(s_t)$ from G_t in (6) changes no expectation. The variance of the per-step term $\nabla \log \pi \cdot (G_t - b)$ is minimized over constant b by a weighted mean of G_t, and over state-dependent b the value $V^\pi(s_t)$ is the choice that removes the part of G_t predictable from the state; a fitted critic approximates it, and its error costs variance, never bias, as long as the fit does not read the action sampled at that step [82].

The generalized advantage telescopes to its two limits. With $\delta_t = R_t + V(s_{t+1}) - V(s_t)$ and $V(s_T) = 0$, $\sum_{l=0}^{T-t-1} \delta_{t+l} = \sum_{u \geq t} R_u - V(s_t) = G_t - V(s_t)$, which is (7) at $\lambda = 1$; at $\lambda = 0$ only δ_t survives. The exponential weights interpolate the two, and the bias of $A_t^{(\lambda)}$ as an estimate of the true advantage is $O(\lambda^k)$ in the critic's error at k steps ahead [78].

PUCT as policy improvement. With exact leaf values and one decision of depth, every backed-up value in (9) is $Q^*(s, a)$ exactly, and as $N(s) \to \infty$ the bonus of every action vanishes relative to the gap between the best mean and the rest, so the most visited action is an argmax of Q^*. At depth the means are of returns under the search's own descents, which concentrate on the children it rates highest; the normalized visit counts are then a distribution whose one-step expected value under Q^π is at least the prior's, which is the policy-improvement step of policy iteration with the prior as π and the visit distribution as the improved policy [82, 80]. The regression suite checks both on the Potts chain: the argmax exactly at depth one, and the improvement on 13 of 14 sampled states at depth three, the exception being a state whose prior already puts most of its mass on an optimal move.

The clipped objective at the collecting policy. With $\pi = \pi_{\text{old}}$, $\rho_t = 1$ for every decision, so both arguments of the minimum in (8) equal A_t and $\nabla\rho_t = \nabla \log \pi(a_t \mid s_t)$; the gradient of (8) is $\sum_t \nabla \log \pi(a_t \mid s_t)\, A_t$, the actor–critic's estimator. Away from the collecting policy the

minimum removes the gradient wherever the ratio has left $[1-\epsilon, 1+\epsilon]$ in the direction the advantage would push it, which is the pessimistic bound on the surrogate improvement that licenses the reuse of a batch [79].

C Derivations of the Bounds

The plug-in bound, (10). $\ell^*(\mathcal{T})$ is a maximum over the feasible set $\boldsymbol{\tau} \geq 0$, and $\hat{\boldsymbol{\tau}}$ lies in it by construction: projected gradient descent on the convex least-squares objective clamps every iterate to the non-negative orthant, so the value at $\hat{\boldsymbol{\tau}}$ is one of the values maximized over. Equality holds exactly when $\hat{\boldsymbol{\tau}}$ is a maximizer.

The parsimony bound, (11). Write $a_e = e^{-kt_e/(k-1)} \in [0,1]$. Each entry of $\mathbf{P}(t_e)$ is affine in a_e and each branch appears exactly once in the product of (18), so the site likelihood is a polynomial of degree at most one in each a_e: multilinear. A multilinear function on a box attains its maximum at a vertex, where each $\mathbf{P}_e$ is either $\mathbf{I}$ (no change permitted) or $\mathbf{J}/k$ (the child's state independent of the parent's, at weight $1/k$). Root the tree at leaf 1, which the pulley principle permits for a reversible model. At a vertex with cut set C (the branches carrying $\mathbf{J}/k$), the site likelihood is a sum over ancestral assignments constant on each component of $\mathcal{T} \setminus C$: $\pi(x_{1,s})\, k^{-|C|}\, k^{\ell'}$, with ℓ' the number of components containing no leaf (each contributes a free sum over k states), provided every leafy component is monochromatic, and zero otherwise. Merging each leafless component into a neighbour across one of its boundary branches removes one cut and one leafless component at a time and leaves a valid partition with $|C| - \ell'$ cuts, all of whose components are leafy and monochromatic; assigning each its leaf state gives an ancestral labelling with at most $|C| - \ell'$ changes, so $F_s \leq |C| - \ell'$ by minimality of the Fitch score. Hence every vertex value is at most $\pi(x_{1,s})\, k^{-F_s}$, and so is the maximum over the box. Summing over sites gives (11); the regression suite enumerates the $2^{|E|}$ vertices at five taxa and finds the bound attained on every site. The entrywise limit $\sup_t \mathbf{P}_{ij}(t)$, one on the diagonal and $1/k$ off it, gives a bound this one dominates, because that matrix is not row-stochastic and the polynomial is increasing in every entry.

The mean-field bound, (12). For any distribution q on configurations, $\log Z = \mathbb{E}_q[-H] + H(q) + \mathrm{KL}(q \,\|\, p)$, and the divergence is non-negative (Gibbs' inequality). Restricting q to products and evaluating the expectation gives the right-hand side; the fixed-point iteration $q_i \propto \exp(h_i + \sum_{j \in \partial i} J_{ij} q_j)$ ascends it coordinate-wise, and the bound holds at every iterate, so a truncated iteration is a looser bound and never a wrong one [53, 56].

The spanning-tree bound, (13). $\log Z(\theta)$ is the log-normalizer of an exponential family and therefore convex in θ. For trees T with weights $\rho(T)$ and parameters $\theta(T)$ with $\sum_T \rho(T)\theta(T) = \theta$, Jensen gives $\log Z(\theta) \leq \sum_T \rho(T) \log Z(\theta(T))$. The implementation takes one tree per edge, built by breadth-first search from that edge so it contains it, weighted uniformly; an edge appearing in a fraction ρ_e of the trees carries J_e/ρ_e on each tree containing it and 0 elsewhere, so the weighted mean is J_e, and the fields are shared. Each $\log Z(\theta(T))$ is exact by the leaf-to-root recursion, in the log domain. The bound is tight when the graph is a tree [87].

The ground-state bracket, (14). $Z(\beta) = \sum_\sigma e^{-\beta H(\sigma)}$ has k^N terms, each at most $e^{-\beta E_{\min}}$ and at least one equal to it, so $-\beta E_{\min} \leq \log Z(\beta) \leq N \log k - \beta E_{\min}$. Combining with $L \leq \log Z(\beta) \leq U$ and dividing by $\beta > 0$ gives the bracket; as β grows the $N \log k$ term is divided away, until the bounds on $\log Z$ themselves loosen.

D Reference Taxonomy

Grouped as the repository groups them, so one routing table serves the code and the documents alike. Within a group, books come first, then reviews, then the papers an algorithm implements or a proposal needs as its oracle.

Infrastructure (build, structure, and speed). Software craft: Martin [54], Blandy et al. [14], Ramalho [70]. Systems and hardware, including memory hierarchies and parallel kernels: Bryant and O'Hallaron [18], Hwu et al. [41]. Python performance: Gorelick and Ozsvald [35], Antão [5].

Optimization (discrete and continuous). Algorithms and discrete mathematics: Cormen et al. [20], Rosen [73], Papadimitriou and Steiglitz [66]; the papers behind the cut-based solvers, Kolmogorov and Zabih [45] on which energies a cut minimizes and Boykov and Kolmogorov [16] on the max-flow they run. Numerical optimization: Nocedal and Wright [63], Boyd and Vandenberghe [15] for the semidefinite certificate; Hansen and Ostermeier [37] for the covariance-adapting evolution strategy proposed as a population baseline. Probabilistic inference and message passing: MacKay [53], Koller and Friedman [44], Frey [29], Ortega [64], Bishop [12], Murphy [58]; the review Wainwright and Jordan [86]; the papers Wainwright et al. [87] and Wainwright et al. [88] for the tree-reweighted bounds, and Hsu et al. [40] for spectral initialization of a hidden Markov model. Statistical physics and Monte Carlo: Mézard and Montanari [56], Newman and Barkema [61], Krauth [48], Landau and Binder [50]; the reviews Betancourt [11] and Schollwöck [75]; the paper Machta [52] for population annealing. Information theory and geometry: Amari [3], Cover and Thomas [21]. Learning and reinforcement learning: Goodfellow et al. [34], Prince [69], Sutton and Barto [82], Lapan [51], Raschka [71]; the papers Sutton et al. [83] and Bacon et al. [9] for options, and Schrittwieser et al. [76] for planning with a learned model. Learning for combinatorial optimization: the reviews Bengio et al. [10] and Mazyavkina et al. [55]; the papers Khalil et al. [42], Kool et al. [46], Paulus et al. [67], Henderson et al. [38], Agarwal et al. [1], and the pair Schuetz et al. [77] and Angelini and Ricci-Tersenghi [4].

Application (the science). Phylogenetics: Felsenstein [26], Durbin et al. [22], Compeau and Pevzner [19], Pachter and Sturmfels [65], Yang [91]; the papers Whidden and Matsen [89], Azouri et al. [8], Zhang and Matsen [93], Zhang and Matsen [94], Speyer and Sturmfels [81], Altekar et al. [2], and the external baselines Minh et al. [57] and Kozlov et al. [47]. Information and coding: Blahut [13], Richardson and Urbanke [72], with Nielsen and Chuang [62] as background only; the papers Gallager [30] and Nachmani et al. [59].